

UNIT III TREE STRUCTURE

Tree ADT – Binary Tree ADT – tree traversals – binary search trees – AVL trees – heaps – multi- way search trees

Binary Tree using Array Representation

Each node contains **info**, **left**, **right** and **father** fields. The left, right and father fields of a node point to the node's left son, right son and father respectively.

Using the array implementation, we may declare,

```
#define NUMNODES 100
struct nodetype
{
    int info;
    int left;
    int right;
    int father;
};
struct nodetype node[NUMNODES];
```

This representation is called linked array representation.

Example: -

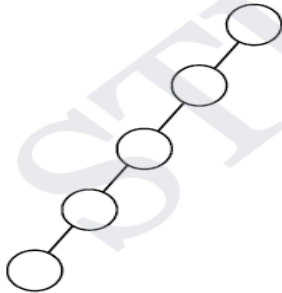


Fig (a)

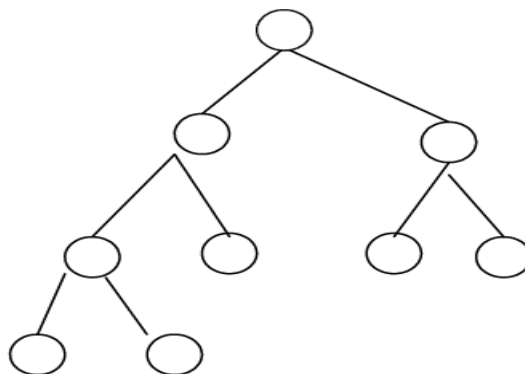


Fig (b)

The above trees can be represented in memory sequentially as follows

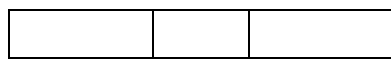
A	
B	
-	
C	
-	
-	
-	
D	
-	
E	

The above representation appears to be good for complete binary trees and wasteful for many other binary trees. In addition, the insertion or deletion of nodes from the middle of a tree requires the insertion of many nodes to reflect the change in level number of these nodes.

Binary Tree using Link Representation

The problems of sequential representation can be easily overcome through the use of a linked representation.

Each node will have three fields LCHILD, DATA and RCHILD as represented below:



LCHILD DATA RCHILD

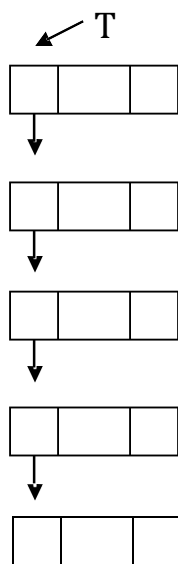


Fig (a)

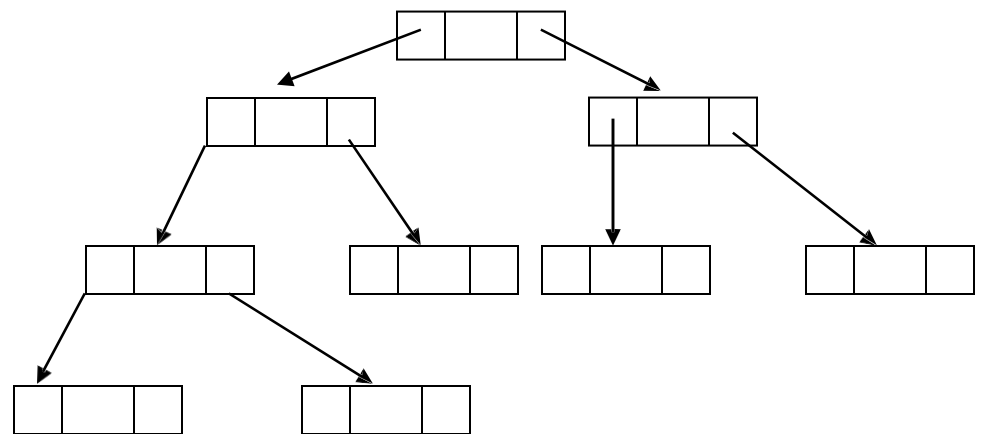


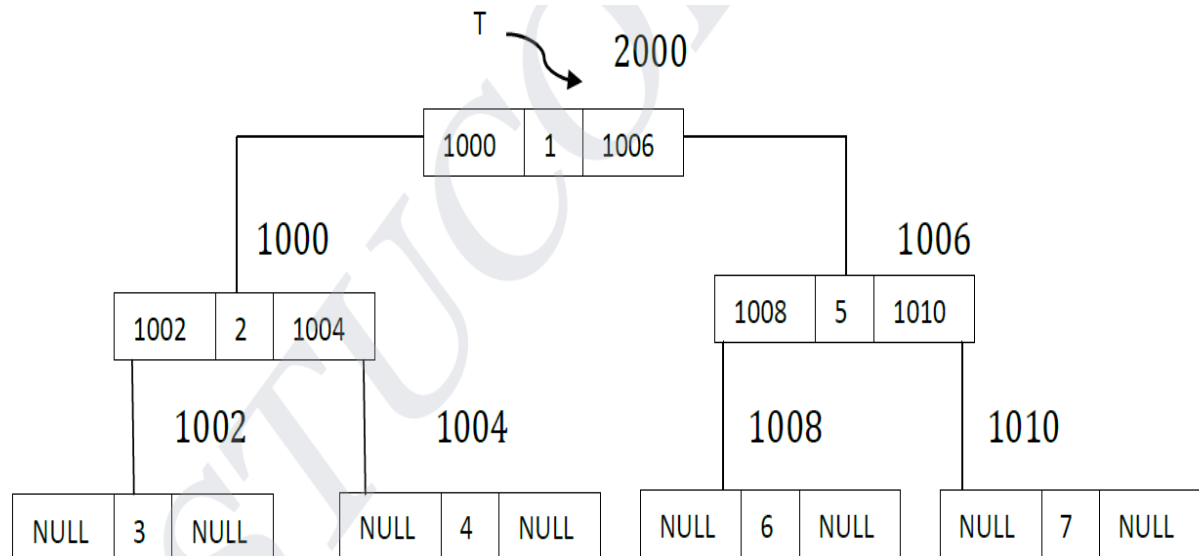
Fig (b)

In most applications it is adequate. But this structure make it difficult to determine the parent of a node since this leads only to the forward movementof the links.

Using the linked implementation we may declare,

Struct treenode

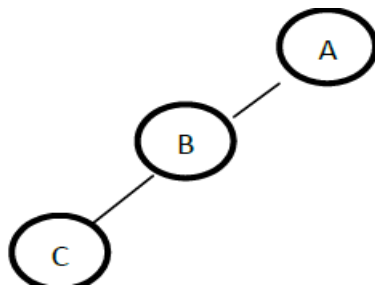
```
{  
int data;  
structtreenode *leftchild;  
structtreenode *rightchild;  
}*T;
```



TYPES OF BINARY TREES

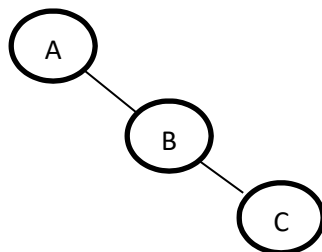
Left Skewed Binary tree :

A binary tree which has only left child is called left skewed binary tree.



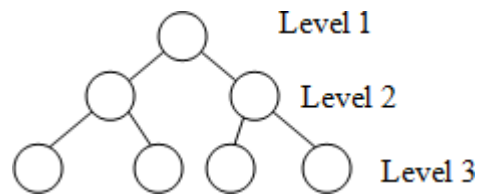
Right Skewed Binary tree :

A Binary tree which has only right child is called right skewed binary tree.



Full Binary Tree :

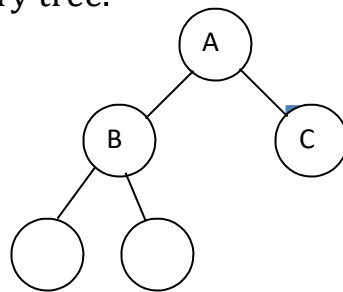
It is the one which has exactly two children for each node at each level and all the leaf nodes should be at the same level.



Complete Binary Tree :

It is the one tree where all the leaf nodes need not be at the same level and at the bottom level of the complete binary tree, the nodes should be filled from the left to the right.

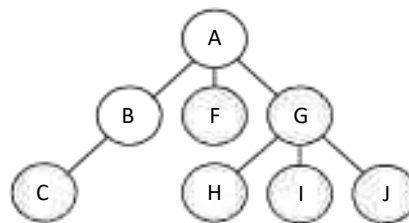
All full binary trees are complete binary tree. But all complete binary trees need not be full binary tree.



CONVERSION OF A GENERAL TREE TO BINARY TREE

General Tree:

A General Tree is a tree in which each node can have an unlimited out degree. Each node may have as many children as is necessary to satisfy its requirements. *Example: Directory Structure*



It is considered easy to represent binary trees in programs than it is to represent general trees. So, the general trees can be represented in binary tree format.

Changing general tree to Binary tree:

The binary tree format can be adopted by changing the meaning of the left and right pointers. There are two relationships in binary tree,

Parent to child

Sibling to sibling

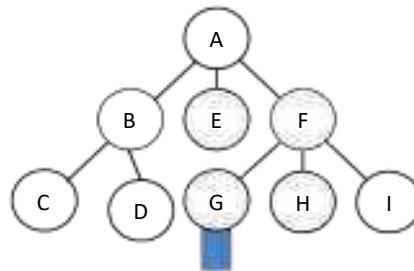
Using these relationships, the general tree can be implemented as binary tree.

Algorithm

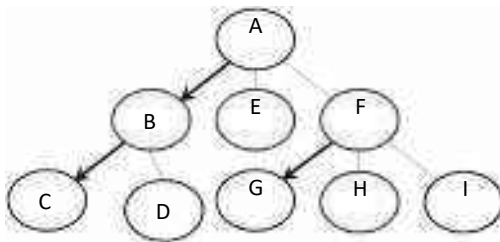
Identify the branch from the parent to its first or leftmost child. These branches from each parent become left pointers in the binary tree

Connect siblings, starting with the leftmost child, using a branch for each sibling to its right sibling.

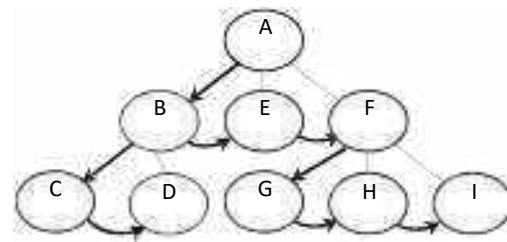
Remove all unconnected branches from the parent to its children



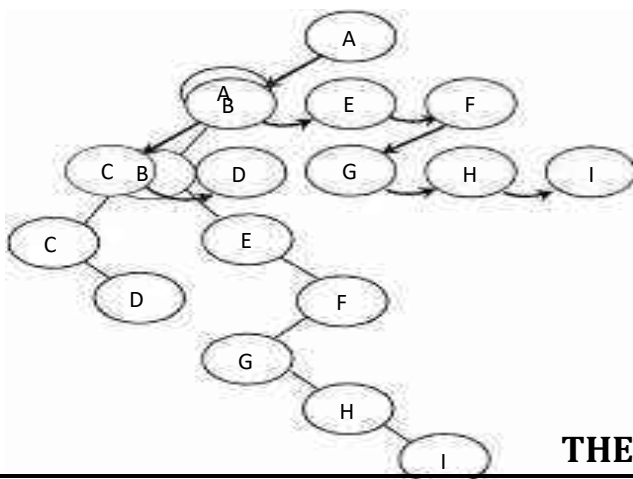
General Tree



Step 1: Identify all leftmost children



Step 2: Connect Siblings



Step 3: Delete unneeded branches

THE RESULTING BINARY TREE

BINARY TREE TRAVERSALS

Compared to linear data structures like linked lists and one dimensional array, which have only one logical means of traversal, tree structures can be traversed in many different ways. Starting at the root of a binary tree, there are three main steps that can be performed and the order in which they are performed defines the traversal type. These steps (in no particular order) are: performing an action on the current node (referred to as "visiting" the node), traversing to the left child node, and traversing to the right child node. Thus the process is most easily described through recursion.

A binary tree traversal requires that each node of the tree be processed once and only once in a predetermined sequence.

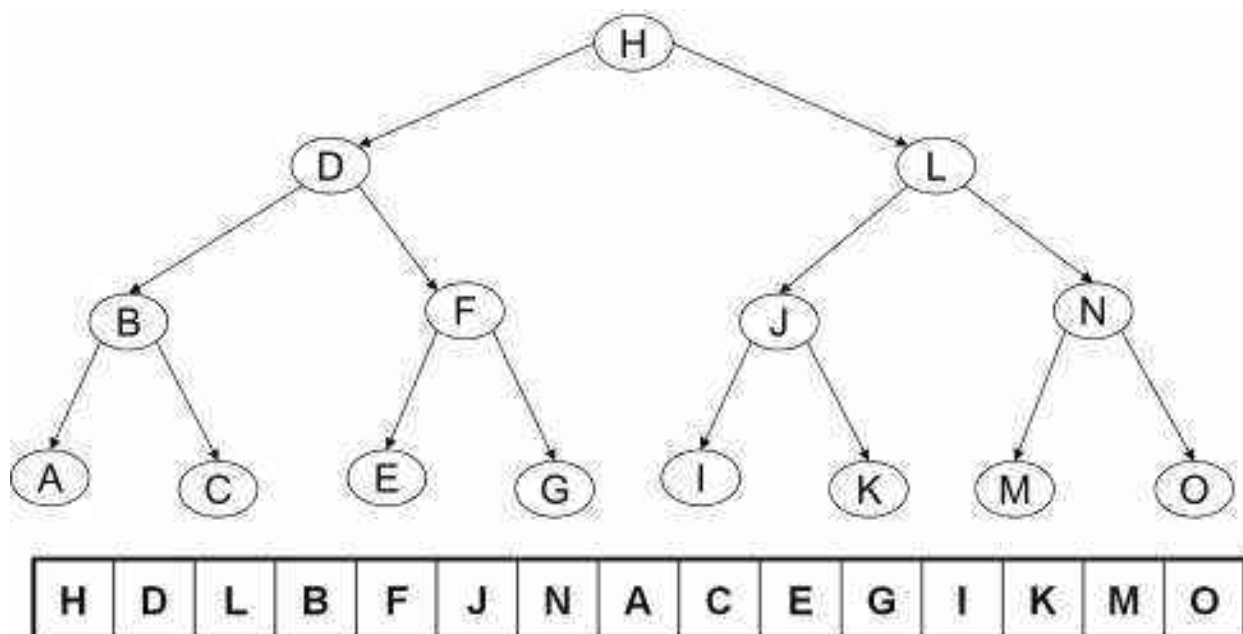
The two general approaches to the traversal sequence are,

Depth first traversal

Breadth first traversal

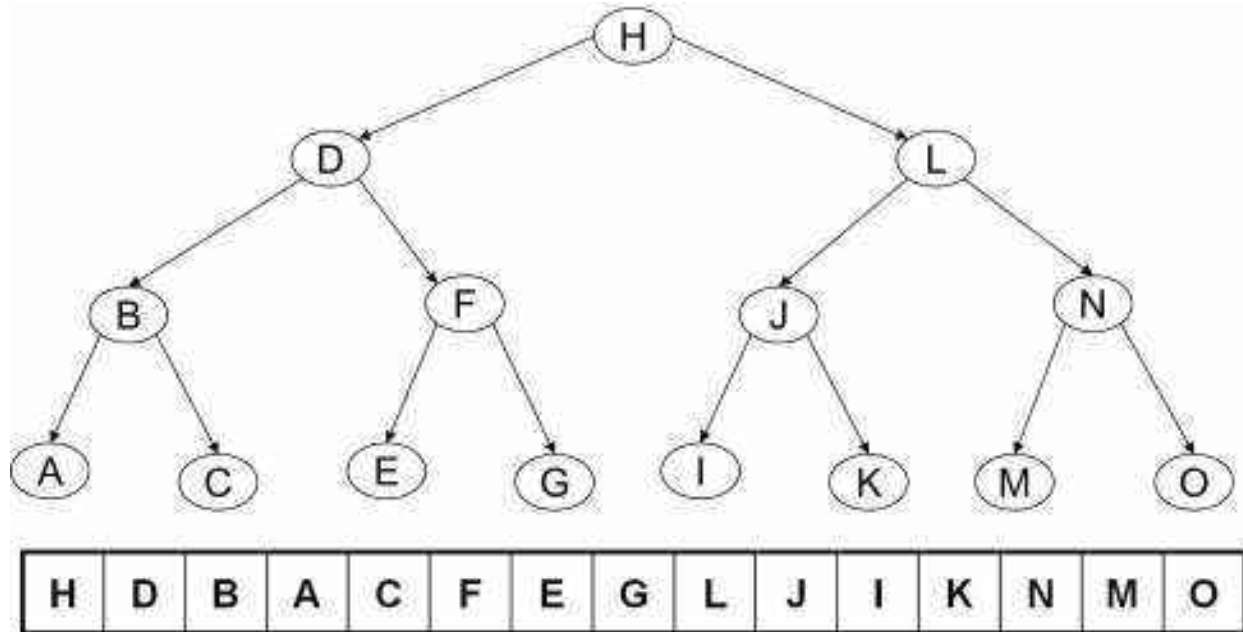
Breadth-First Traversal

In a breadth-first traversal, the processing proceeds horizontally from the root to all its children, then to its children's children, and so forth until all nodes have been processed. In other words, in breadth traversal, each level is completely processed before the next level is started.



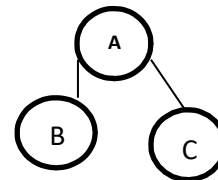
Depth-First Traversal

In depth first traversal, the processing proceeds along a path from the root through one child to the most distant descendent of that first child before processing a second child. In other words, in the depth first traversal, all the descendants of a child are processed before going to the next child.



There are basically three ways of binary tree traversals.

1. **Inorder** --- (left child,root,right child)
2. **Preorder** --- (root,left child,right child)
3. **Postorder** --- (left child,right child,root)



Inorder--- B A C

Preorder --- A B C

Postorder --- B C A

In C, each node is defined as a structure of the following form:

```
struct node
{
    int info;
    struct node *lchild;
    struct node *rchild;
}
typedef struct node NODE;
```

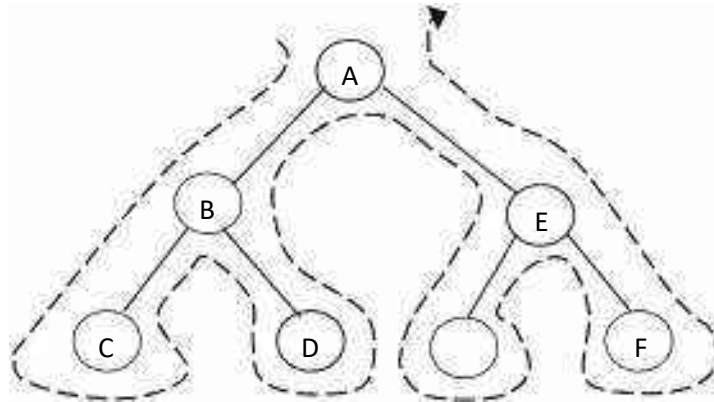
Inorder Traversal

Steps :

Traverse left subtree in inorder

Process root node

Traverse right subtree in inorder



The Output is : $C \rightarrow B \rightarrow D \rightarrow A \rightarrow E \rightarrow F$

Algorithm

Algorithm inorder traversal (BinTree T)

Begin

If (not empty (T)) then

Begin

Inorder_traversal (left subtree (T))

Print (info (T)) /* process node */

Inorder_traversal (right subtree (T))

End

End

Routines

void inorder_traversal (NODE * T)

{

if(T != NULL)

{

```

inorder_traversal(T->lchild);
printf("%d \t ", T->info);
inorder_traversal(T->rchild);
}
}

```

Preorder Traversal

Steps :

Process root node

Traverse left subtree in preorder

Traverse right subtree in preorder

Algorithm

Algorithm inorder traversal (BinTree T)

Begin

If (not empty (T)) then

Begin

Print (info (T)) / * process node */

Preorder_traversal (left subtree (T))

Preorder_traversal (right subtree (T))

End

End

Routines

void inorder_traversal (NODE * T)

{

if(T != NULL)

{

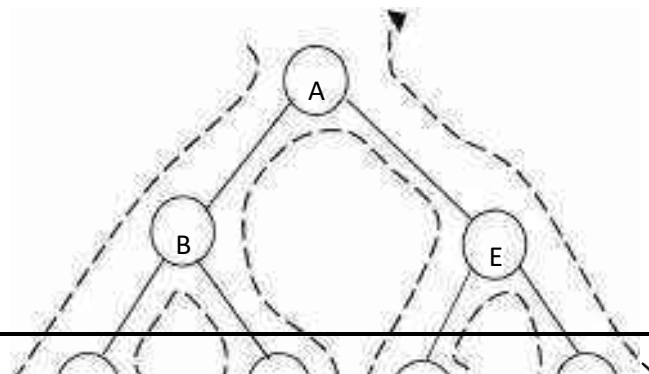
printf("%d \t ", T->info);

preorder_traversal(T->lchild);

preorder_traversal(T->rchild);

}

}



Output is : $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$

Postorder Traversal

Steps :

Traverse left subtree in postorder
Traverse right subtree in postorder
process root node

Algorithm

Algorithm postorder traversal (BinTree T)

Begin

If (not empty (T)) then

Begin

Postorder_traversal (left subtree (T))

Postorder_traversal (right subtree(T))

Print (Info (T)) / * process node */

End

End

Routines

void postorder_traversal (NODE * T)

{

if(T != NULL)

{

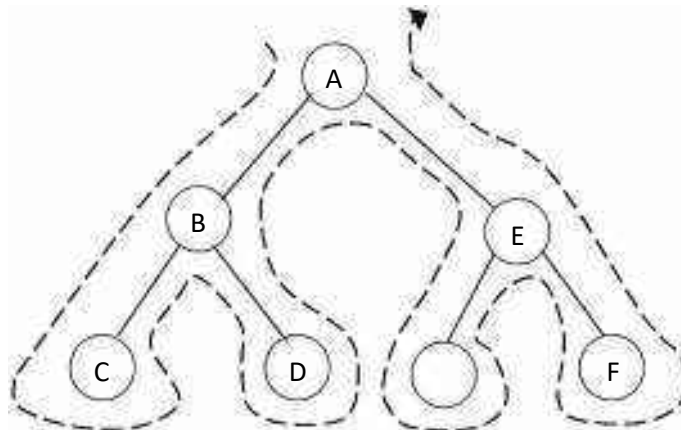
postorder_traversal(T->lchild);

postorder_traversal(T->rchild);

printf("%d \t", T->info);

}

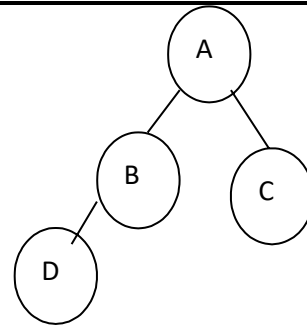
}



Output is : $C \rightarrow D \rightarrow B \rightarrow F \rightarrow E \rightarrow A$

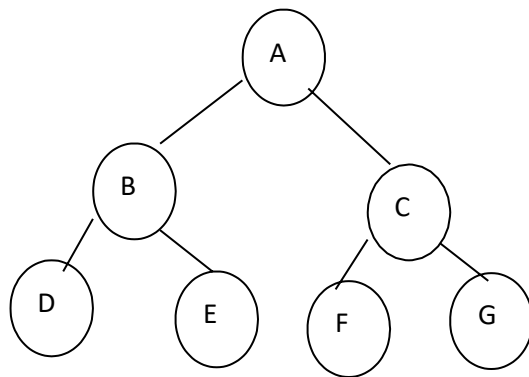
Examples :

1.FIND THE TRAVERSAL OF THE FOLLOWING TREE



ANSWER : POSTORDER: DBCA INORDER: DBAC PREORDER:ABCD

2.FIND THE TRAVERSAL OF THE FOLLOWING TREE



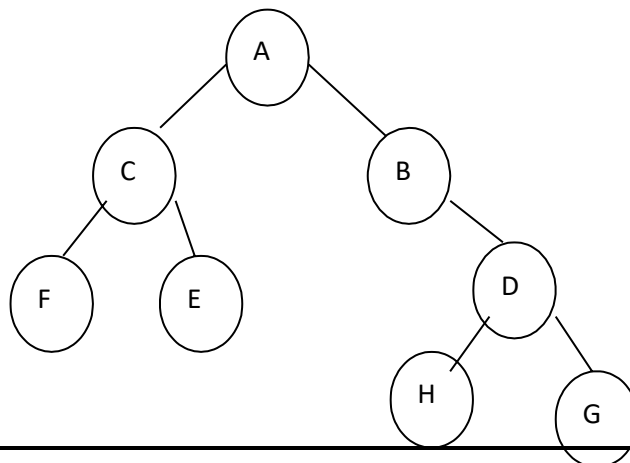
ANSWER : POSTORDER: DEBFGCA INORDER: DBEAFCG
PREORDER:ABDECFG

3.A BINARY TREE HAS 8 NODES. THE INORDER AND POSTORDER TRAVERSAL OF THE TREE ARE GIVEN BELOW. DRAW THE TREE AND FIND PREORDER.

POSTORDER: F E C H G D B A

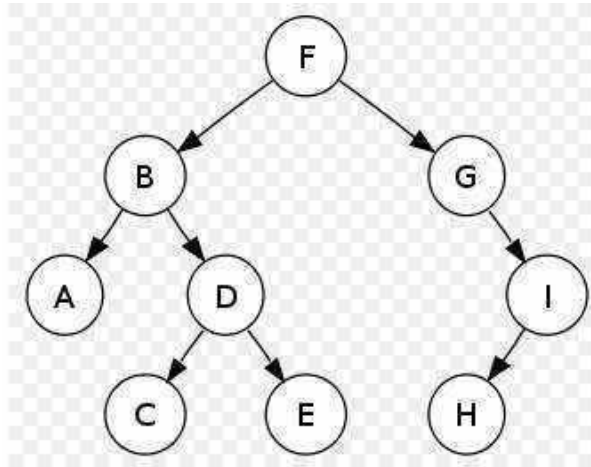
INORDER: F C E A B H D G

Answer:



PREORDER: ACFEBDHG

Example 4



Preorder traversal sequence: F, B, A, D, C, E, G, I, H (root, left, right)

Inorder traversal sequence: A, B, C, D, E, F, G, H, I (left, root, right)

Postorder traversal sequence: A, C, E, D, B, H, I, G, F (left, right, root)

APPLICATIONS

1. Some applications of preorder traversal are the evaluation of expressions in prefix notation and the processing of abstract syntax trees by compilers.
2. Binary search trees (a special type of BT) use inorder traversal to print all of their data in alphanumeric order.
3. A popular application for the use of postorder traversal is the evaluating of expressions in postfix notation.

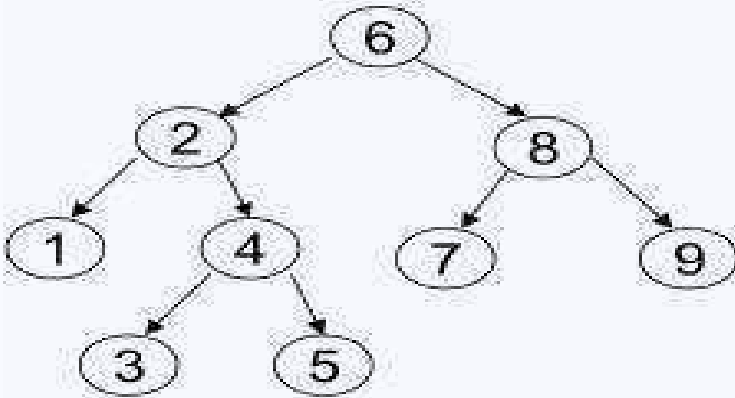
BINARY SEARCH TREE

Binary search tree (BST) is a node-based binary tree data structure which has the following properties:

- The left sub-tree of a node contains only nodes with keys less than the node's key.
- The right sub-tree of a node contains only nodes with keys greater than the node's key.
- Both the left and right sub-trees must also be binary search trees.

From the above properties it naturally follows that:

Each node (item in the tree) has a distinct key.



Program: Creating a Binary Search Tree

We assume that every node of a binary search tree is capable of holding an integer data item and that the links can be made to point to the root of the left subtree and the right subtree, respectively. Therefore, the structure of the node can be defined using the following declaration:

```
struct tnode
{
    int data;
    struct tnode *lchild,*rchild;
};
```

A complete C program to create a binary search tree follows:

```
#include <stdio.h>
#include <stdlib.h>
struct tnode
{
    int data;
    struct tnode *lchild, *rchild;
};

struct tnode *insert(struct tnode *p,int val)
{
    struct tnode *temp1,*temp2;
    if(p == NULL)
    {
```

```

p = (struct tnode *) malloc(sizeof(struct tnode)); /* insert the new node as
root node*/
if(p == NULL)
{
printf("Cannot allocate\n");
exit(0);
}
p->data = val;
p->lchild=p->rchild=NULL;
}
else
{
temp1 = p;
/* traverse the tree to get a pointer to that node whose child will be the newly
created node*/
while(temp1 != NULL)
{
temp2 = temp1;
if( temp1 ->data > val)
temp1 = temp1->lchild;
else
temp1 = temp1->rchild;
}
if( temp2->data > val)
{
temp2->lchild = (struct tnode*)malloc(sizeof(struct tnode));/*inserts the
newly created node as left child*/
temp2 = temp2->lchild;
if(temp2 == NULL)
{
printf("Cannot allocate\n");
exit(0);
}
temp2->data = val;
temp2->lchild=temp2->rchild = NULL;
}
else
{

```

```

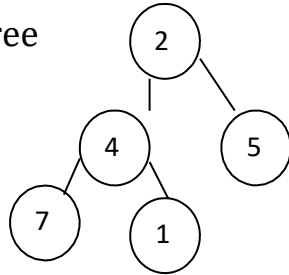
temp2->rchild = (struct tnode*)malloc(sizeof(struct tnode));/* inserts the
newly created node
as left child*/
temp2 = temp2->rchild;
if(temp2 == NULL)
{
printf("Cannot allocate\n");
exit(0);
}
temp2->data = val;
temp2->lchild=temp2->rchild = NULL;
}
}
return(p);
}
/* a function to binary tree in inorder */
void inorder(struct tnode *p)
{
if(p != NULL)
{
inorder(p->lchild);
printf("%d\t",p->data);
inorder(p->rchild);
}
}
void main()
{
struct tnode *root = NULL;
int n,x;
printf("Enter the number of nodes\n");
scanf("%d",&n);
while( n > 0)
{
printf("Enter the data value\n");
scanf("%d",&x);
root = insert(root,x);
}
inorder(root);

```

}

EXAMPLE Construct a BST with nodes 2,4,5,7,1

Normal Tree

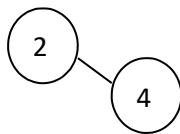


Binary Search Tree

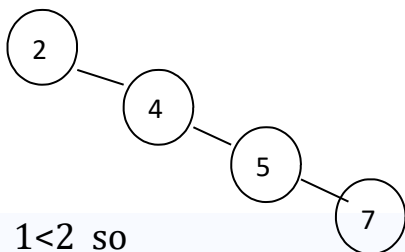
- ✓ The Values in the left subtree must be smaller than the keyvalue to be inserted.
- ✓ The Values in the right subtree must be larger than the keyvalue to be inserted.

Take the 1st element 2 and compare with 4. $2 < 4$

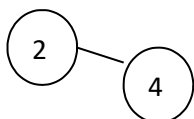
So

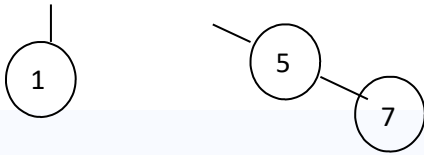


Similarly $2 < 5, 5 > 4$ and $7 > 2, 7 > 4, 7 > 5$



and $1 < 2$ so





is the final BST.

OPERATIONS

- ✓ Operations on a binary tree require comparisons between nodes. These
- ✓ comparisons are made with calls to a comparator, which is a subroutine that computes the total order (linear order) on any two values. This
- ✓ comparator can be explicitly or implicitly defined, depending on the language in which the BST is implemented.
- ✓ The following are the operations that are being done in Binary Tree
 - Searching.
 - Sorting.
 - Deletion.
 - Insertion.

Binary search tree declaration routine

```
Struct treenode;  
Typedef struct treenode *position;  
Typedef struct treenode *searchtree;  
Typedef int elementtype;  
Struct treenode  
{  
    Elementtype element;  
    Searchtree left;  
    Searchtree right;  
};  
Struct treenode  
{  
    int element;  
    struct treenode *left;  
    struct treenode *right;
```

```
};
```

Make_null

This operation is mainly for initialization. Some programmers prefer to initialize the first element as a one-node tree, but our implementation follows the recursive definition of trees more closely. It is also a simple routine.

Routine to make an empty tree

```
SEARCH_TREE  
make_null ( void )  
{  
    return NULL;  
}
```

Find

This operation generally requires returning a pointer to the node in tree T that has key x , or $NULL$ if there is no such node. The structure of the tree makes this simple. If T is $NULL$, then we can just return $NULL$. Otherwise, if the key stored at T is x , we can return T . Otherwise, we make a recursive call on a subtree of T , either left or right, depending on the relationship of x to the key stored in T . The code in Figure 4.18 is an implementation of this strategy.

Find operation for binary search trees

```
Position find(struct treenode T, int num)  
{  
    While(T!=NULL)  
    {  
        if(num>T->data)  
        {  
            T=T->right;  
        }  
        if(num<T->data)  
        {  
            T=T->left;  
        }  
    }  
    return T;  
}
```

```

T=T-->left;
}
else if(num< T-->data)
{
T=T-->left;
if(num>T-->data)
T=T-->right;
}
if(T-->data==num)
break;
}
return T;
}
// To find a Number
Position find(elementtype X, searchtree T)
{
If(T==NULL)
return NULL;
if(x< T-->element)
return find(x,T-->left);
else if(X> T-->element)
return find(X,T-->right);
else
return T;
}

```

Find_min and Find_max

Recursive implementation of find_min & find_max for binary search trees

```

// Finding Minimum
Position findmin(searchtree T)
{
if(T==NULL)

```

```

return NULL;
else if(T-->left==NULL)
return T;
else return findmin(T-->left);
}
// Finding Maximum
Position findmax(searchtree T)
{
if(T==NULL)
return NULL;
else if(T-->right==NULL)
return T;
else return findmin(T-->right);
}

```

Nonrecursive implementation of find_min & find_max for binary search trees

```

// Finding Maximum
Position findmax(searchtree T)
{
if( T!=NULL)
while(T-->Right!=NULL)
T=T-->right;
Return T;
}
// Finding Minimum
Position findmin(searchtree T)
{
if( T!=NULL)
while(T-->left!=NULL)
T=T-->left;
Return T;
}

```

Insert

The insertion routine is conceptually simple. To insert x into tree T , proceed down the tree as you would with a *find*. If x is found, do nothing (or "update" something). Otherwise, insert x at the last spot on the path traversed. Figure below shows what happens. To insert 5, we traverse the tree as though a *find* were occurring. At the node with key 4, we need to go right, but there is no subtree, so 5 is not in the tree, and this is the correct spot.

Duplicates can be handled by keeping an extra field in the node record indicating the frequency of occurrence. This adds some extra space to the entire tree, but is better than putting duplicates in the tree (which tends to make the tree very deep). Of course this strategy does not work if the key is only part of a larger record. If that is the case, then we can keep all of the records that have the same key in an auxiliary data structure, such as a list or another search tree.

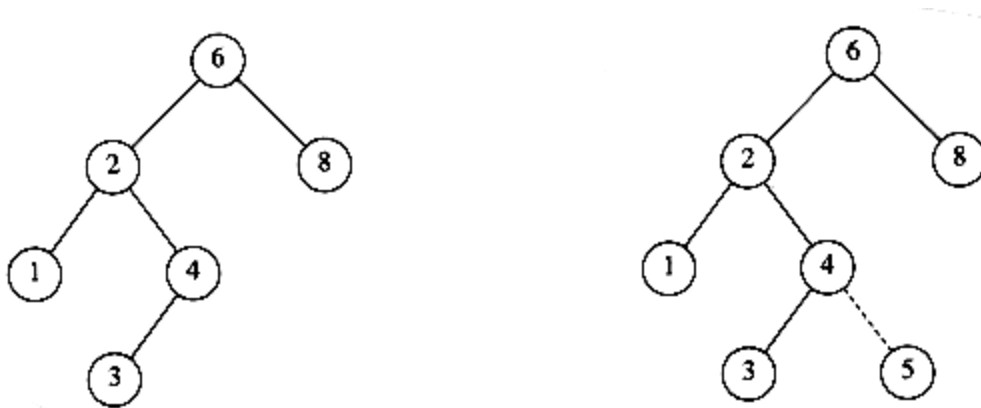


Figure shows the code for the insertion routine. Since T points to the root of the tree, and the root changes on the first insertion, *insert* is written as a function that returns a pointer to the root of the new tree. Lines 8 and 10 recursively insert and attach x into the appropriate subtree.

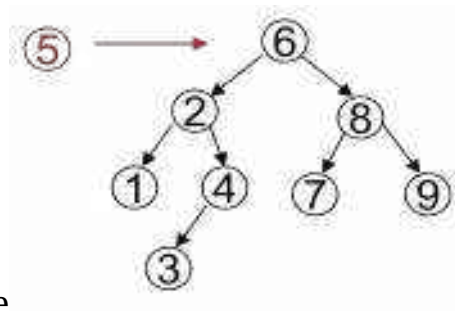
Insertion into a binary search tree

```
Searchtree insert(elementtype X, Searchtree T)
```

```

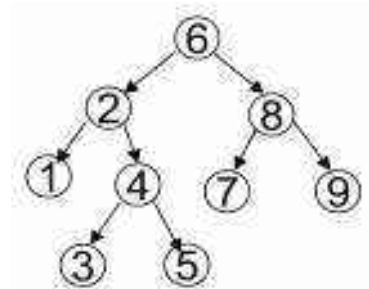
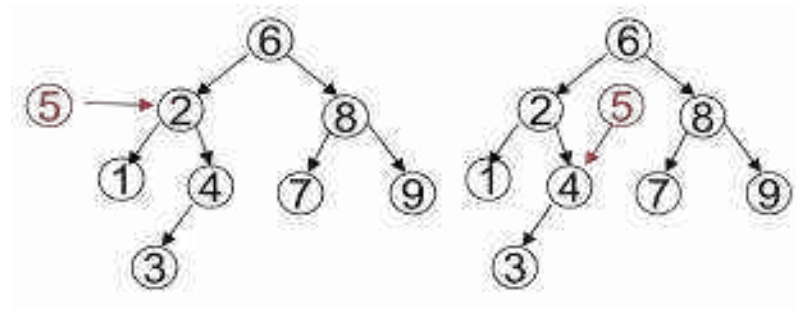
{
If(T== NULL)
{
/* create and return a one node tree*/
T=malloc(sizeof(structtreenode));
If(T==NULL)
Fatalerror("Out of Space");
Else
{
T-->element=X;
T-->left=T-->right=NULL;
}
}
Else if(x<T-->element)
T-->left=insert(X,T-->left);
Else if(X>=T-->left)
T-->right=insert(X,T-->right);
Return T;
}

```



EXAMPLE Insert node 5 in given tree

STEP 1: Now $5 < 6$ and $5 > 2$ and $5 < 4$ so



Thus 5 is inserted.

Delete

As is common with many data structures, the hardest operation is deletion. Once we have found the node to be deleted, we need to consider several possibilities.

If the node is a leaf, it can be deleted immediately. If the node has one child, the node can be deleted after its parent adjusts a pointer to bypass the node (we will draw the pointer directions explicitly for clarity).. Notice that the deleted node is now unreferenced and can be disposed of only if a pointer to it has been saved. The complicated case deals with a node with two children. The general strategy is to replace the key of this node with the smallest key of the right subtree (which is easily found) and recursively delete that node (which is now empty). Because the smallest node in the right subtree cannot have a left child, the second *delete* is an easy one.

To delete an element, consider the following three possibilities :

Case 1: Node to be deleted is a leaf node.

Case 2: Node with only one child.

Case 3: Node with two children.

Case 1: Node with no children | Leaf node :

1. Search the parent of the leaf node and make the link to the leaf node as NULL.

2. Release the memory of the deleted node.

Case 2: Node with only one child :

1. Search the parent of the node to be deleted.

2. Assign the link of the parent node to the child of the node to be deleted.

3. Release the memory for the deleted node.

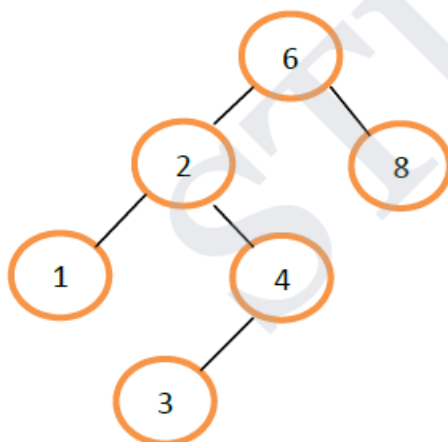
Case 3: Node with two children :

It is difficult to delete a node which has two children.

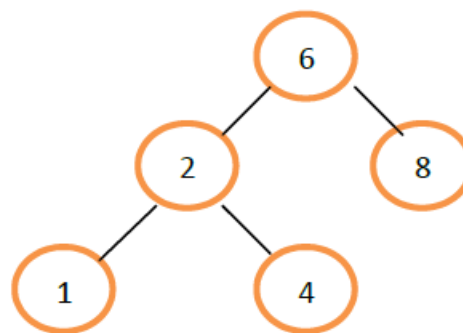
So, a general strategy has to be followed.

1. Replace the data of the node to be deleted with either the largest element from the left subtree or the smallest element from the right subtree.

Case 1:

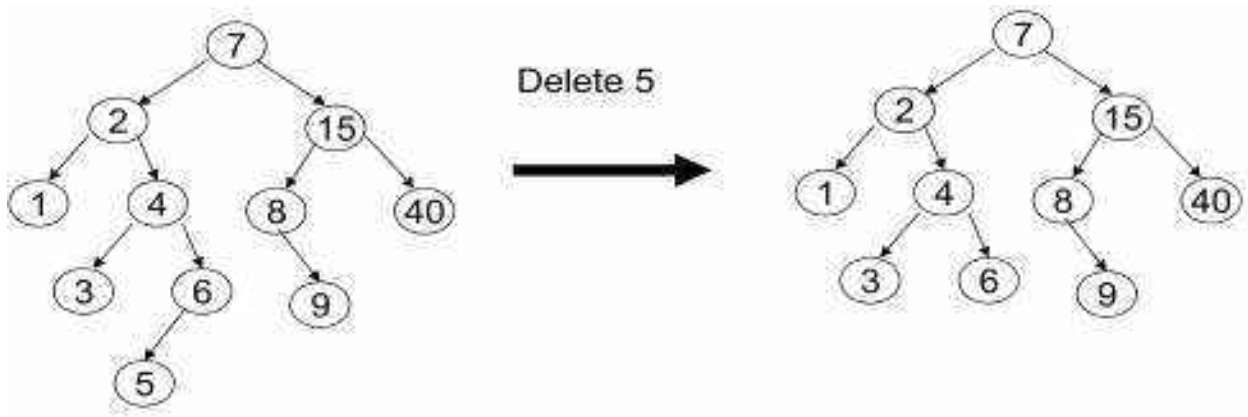


Before Deletion

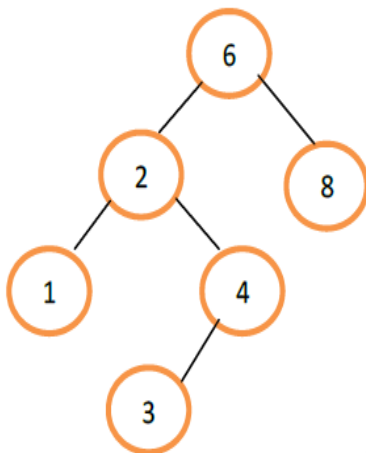


After Deletion of 3

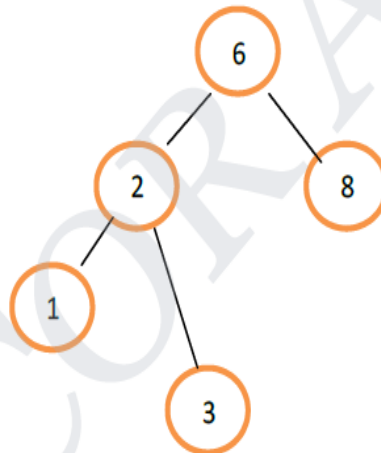
EXAMPLE :



Case 2 :

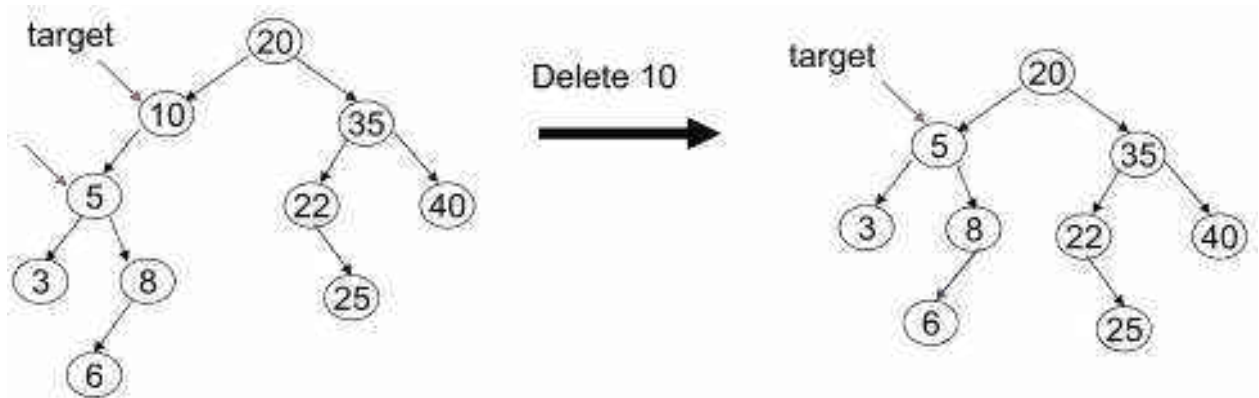


Before deletion of 4

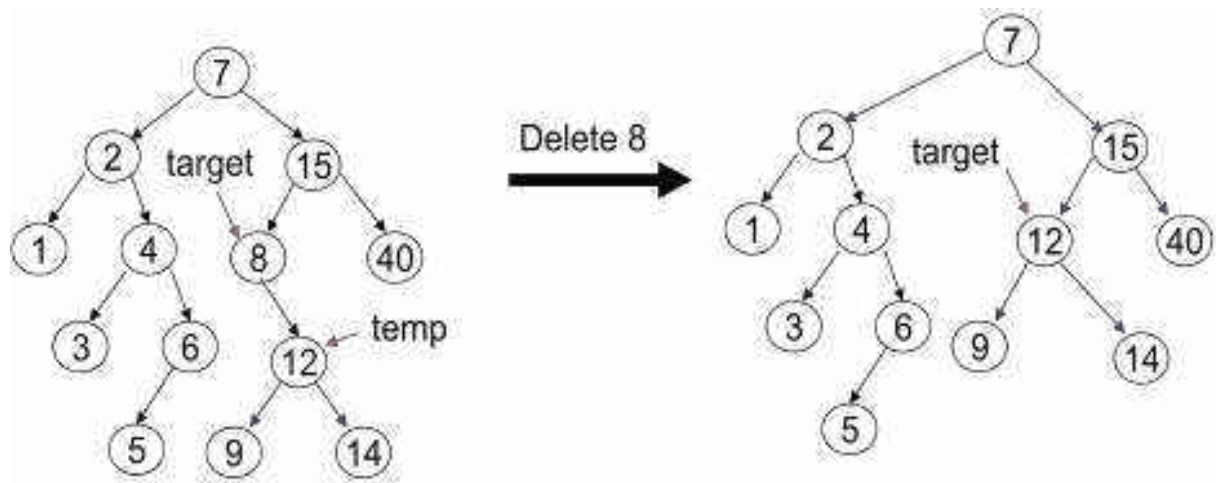


After deletion of 4

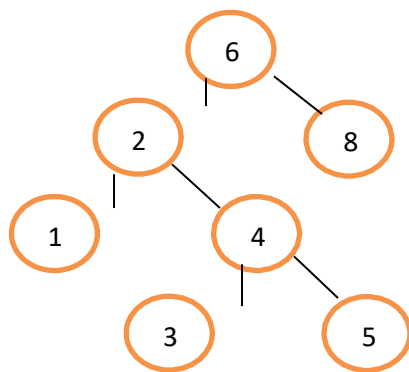
EXAMPLE The right subtree of the node x to be deleted is empty.



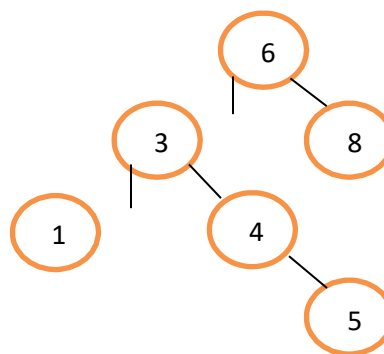
EXAMPLE The left subtree of the node x to be deleted is empty.



Case 3 :



Before deletion of 2

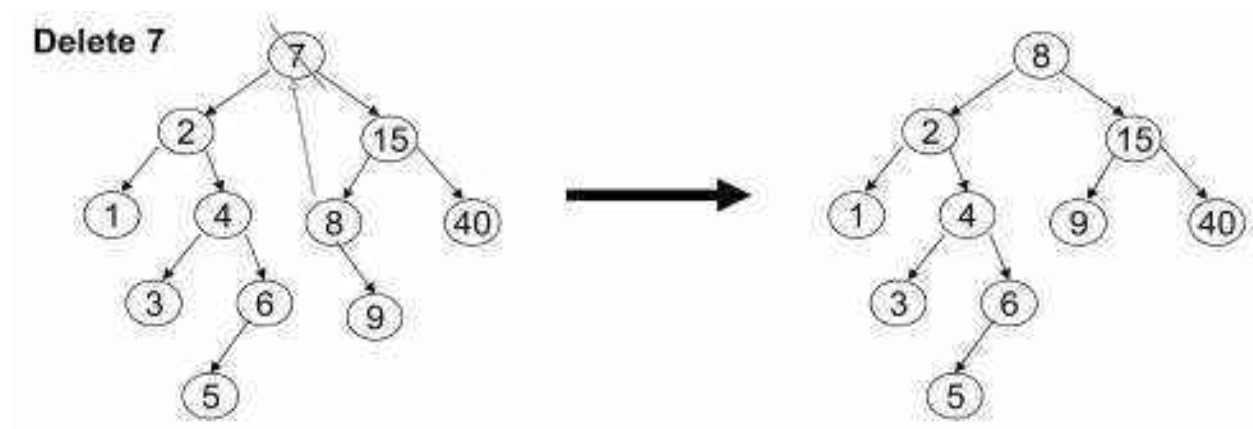


After deletion

EXAMPLE

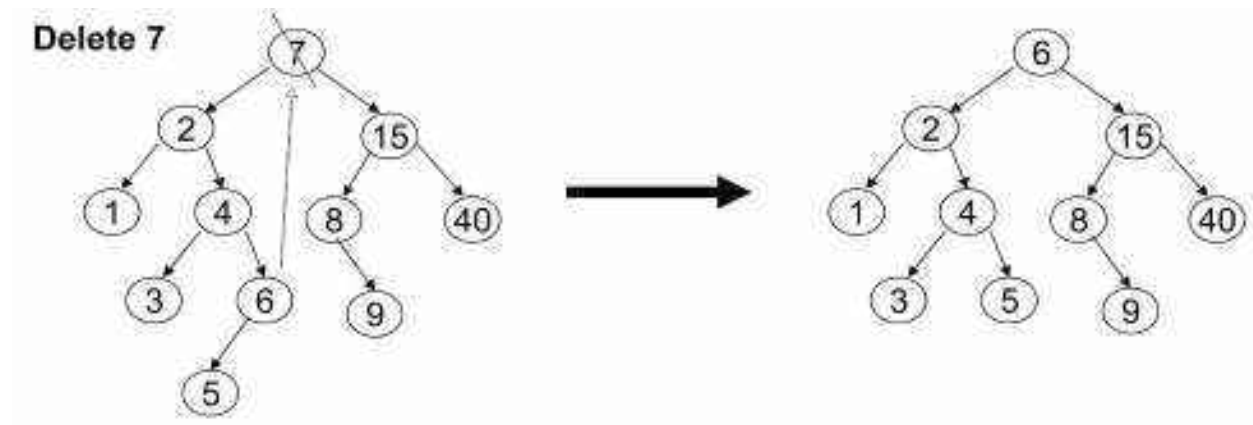
DELETION BY COPYING: METHOD#1

Copy the minimum key in the right subtree of x to the node x, then delete the one-child or leaf-node with this minimum key.



DELETION BY COPYING: METHOD#2

Copy the **maximum** key in the **left** subtree of x to the node x, then delete the one-child or leaf-node with this **maximum** key.



The code in performs deletion. It is inefficient, because it makes two passes down the tree to find and delete the smallest node in the right subtree when

this is appropriate. It is easy to remove this inefficiency, by writing a special *delete_min* function, and we have left it in only for simplicity.

If the number of deletions is expected to be small, then a popular strategy to use is *lazy deletion*: When an element is to be deleted, it is left in the tree and merely *marked* as being deleted. This is especially popular if duplicate keys are present, because then the field that keeps count of the frequency of appearance can be decremented. If the number of real nodes in the tree is the same as the number of "deleted" nodes, then the depth of the tree is only expected to go up by a small constant (why?), so there is a very small time penalty associated with lazy deletion. Also, if a deleted key is reinserted, the overhead of allocating a new cell is avoided.

Deletion routine for binary search trees

```
Searchtree delete(elementtype X, searchtree T)
{
    positiontmpcell;
    if(T==NULL)
        error("element not found");
    else if(X<T-->element)
        T-->left=delete(X,T-->left);
    Else if(X>T-->element)
        T-->right=delete(X,T-->right);
    Else if(T-->left != NULL && T-->right!=NULL)
    {
        /* Replace with smallest in right subtree*/
        Tmpcell=findmin(T-->right);
        T-->element=tmpcell-->element;
        T-->right=delete(T-->element,T-->right);
    }
    Else
    {
        /* One or Zero children*/
        tmpcell=T;
        if(T-->left==NULL)
```

```

T=T-->right;
Else if(T-->right==NULL)
T=T-->left;
Free(tmpcell);
}
Return T;
}

```

COUNTING THE NUMBER OF NODES IN A BINARY SEARCH TREE

Introduction

To count the number of nodes in a given binary tree, the tree is required to be traversed recursively until a leaf node is encountered. When a leaf node is encountered, a count of 1 is returned to its previous activation (which is an activation for its parent), which takes the count returned from both the children's activation, adds 1 to it, and returns this value to the activation of its parent. This way, when the activation for the root of the tree returns, it returns the count of the total number of the nodes in the tree.

Program

A complete C program to count the number of nodes is as follows:

```

#include <stdio.h>
#include <stdlib.h>
struct tnode
{
int data;
struct tnode *lchild, *rchild;
};
int count(struct tnode *p)
{
if( p == NULL)
return(0);
else

```

```

if( p->lchild == NULL && p->rchild == NULL)
return(1);
else
return(1 + (count(p->lchild) + count(p->rchild)));

}

struct tnode *insert(struct tnode *p,int val)
{
struct tnode *temp1,*temp2;
if(p == NULL)
{
p = (struct tnode *) malloc(sizeof(struct tnode)); /* insert the new node as root
node*/
if(p == NULL)
{
printf("Cannot allocate\n");
exit(0);
}
p->data = val;
p->lchild=p->rchild=NULL;
}
else
{
temp1 = p;
/* traverse the tree to get a pointer to that node whose child will be the newly
created node*/
while(temp1 != NULL)
{
temp2 = temp1;
if( temp1 ->data > val)
temp1 = temp1->lchild;
else
temp1 = temp1->rchild;
}
if( temp2->data > val)
{
temp2->lchild = (struct tnode*)malloc(sizeof(struct tnode)); /

```

```

*inserts the newly created node
as left child*/
temp2 = temp2->lchild;
if(temp2 == NULL)
{
printf("Cannot allocate\n");
exit(0);
}
temp2->data = val;
temp2->lchild=temp2->rchild = NULL;
}
else
{
temp2->rchild = (struct tnode*)malloc(sizeof(struct tnode));/* inserts the
newly created node
as left child*/
temp2 = temp2->rchild;
if(temp2 == NULL)
{
printf("Cannot allocate\n");
exit(0);
}
temp2->data = val;
temp2->lchild=temp2->rchild = NULL;
}
}
return(p);
}
/* a function to binary tree in inorder */
void inorder(struct tnode *p)
{
if(p != NULL)
{
inorder(p->lchild);
printf("%d\t",p->data);
inorder(p->rchild);
}
}
}

```

```

void main()
{
    struct tnode *root = NULL;
    int n,x;
    printf("Enter the number of nodes\n");
    scanf("%d",&n);
    while( n --- > 0)
    {
        printf("Enter the data value\n");
        scanf("%d",&x);
        root = insert(root,x);
    }
    inorder(root);
    printf("\nThe number of nodes in tree are :%d\n",count(root));
}

```

Explanation

- ✓ Input:
 - 1.The number of nodes that the tree to be created should have
 - 2. The data values of each node in the tree to be created
- ✓ Output:
 - The data value of the nodes of the tree in inorder
 - 2. The count of number of node in a tree.

Example

- ✓ Input:
 - 1.The number of nodes the created tree should have = 5
 - 2. The data values of nodes in the tree to be created are: 10, 20, 5, 9, 8
- ✓ Output: 1. 5 8 9 10 20
 - 2. The number of nodes in the tree is 5

SWAPPING OF LEFT & RIGHT SUBTREES OF A GIVEN BINARY TREE

Introduction

An elegant method of swapping the left and right subtrees of a given binary tree makes use of a recursive algorithm, which recursively swaps the left and right subtrees, starting from the root.

Program

```
#include <stdio.h>
#include <stdlib.h>
struct tnode
{
    int data;
    struct tnode *lchild, *rchild;
};

struct tnode *insert(struct tnode *p, int val)
{
    struct tnode *temp1, *temp2;
    if(p == NULL)
    {
        p = (struct tnode *) malloc(sizeof(struct tnode)); /* insert the new node as
        root node*/
        if(p == NULL)
        {
            printf("Cannot allocate\n");
            exit(0);
        }
        p->data = val;
        p->lchild = p->rchild = NULL;
    }
    else
    {
        temp1 = p;
        /* traverse the tree to get a pointer to that node whose child will be the newly
        created node*/
        while(temp1 != NULL)
        {
            temp2 = temp1;
            if( temp1 ->data > val)
                temp1 = temp1->lchild;
```

```

else
temp1 = temp1->rchild;
}
if( temp2->data > val)
{
temp2->lchild = (struct tnode*)malloc(sizeof(struct tnode));/* inserts the
newly created node
as left child*/
temp2 = temp2->lchild;
if(temp2 == NULL)
{
printf("Cannot allocate\n");
exit(0);
}
temp2->data = val;
temp2->lchild=temp2->rchild = NULL;
}
else
{
temp2->rchild = (struct tnode*)malloc(sizeof(struct tnode));/* inserts the
newly created node
as left child*/
temp2 = temp2->rchild;
if(temp2 == NULL)
{
printf("Cannot allocate\n");
exit(0);
}
temp2->data = val;
temp2->lchild=temp2->rchild = NULL;
}
}
return(p);
}
/* a function to binary tree in inorder */
void inorder(struct tnode *p)
{
if(p != NULL)

```

```

{
inorder(p->lchild);
printf("%d\t",p->data);
inorder(p->rchild);
}
}
struct tnode *swaptree(struct tnode *p)
{
struct tnode *temp1=NULL, *temp2=NULL;
if( p != NULL)
{ temp1= swaptree(p->lchild);
temp2 = swaptree(p->rchild);
p->rchild = temp1;
p->lchild = temp2;
}
return(p);
}
void main()
{
struct tnode *root = NULL;
int n,x;
printf("Enter the number of nodes\n");
scanf("%d",&n);
while( n > 0)
{
printf("Enter the data value\n");
scanf("%d",&x);
root = insert(root,x);
}
printf("The created tree is :\n");
inorder(root);
printf("The tree after swapping is :\n");
root = swaptree(root);
inorder(root);
printf("\nThe original tree is \n");
root = swaptree(root);
inorder(root);
}

```

Explanation

- Input:
 - 1.The number of nodes that the tree to be created should have
 - 2. The data values of each node in the tree to be created
- Output:
 - 1.The data value of the nodes of the tree in inorder before interchanging the left and right subtrees
 - 2. The data value of the nodes of the tree in inorder after interchanging the left and right subtrees

Example

- Input:
 - 1.The number of nodes that the created tree should have = 5
 - 2. The data values of the nodes in tree to be created are: 10, 20, 5, 9, 8
- Output:
 - 1. 5 8 9 10 20
 - 2. 20 10 9 8 5

Applications of Binary Search Trees

One of the applications of a binary search tree is the implementation of a dynamic dictionary. This application is appropriate because a dictionary is an ordered list that is required to be searched frequently, and is also required to be updated (insertion and deletion mode) frequently. So it can be implemented by making the entries in a dictionary into the nodes of a binary search tree. A more efficient implementation of a dynamic dictionary involves considering a key to be a sequence of characters, and instead of searching by comparison of entire keys, we use these characters to determine a multi-way branch at each step. This will allow us to make a 26-way branch according to the first letter, followed by another branch according to the second letter and so on.

Applications of Trees

1. Compiler Design.
2. Unix / Linux.
3. Database Management.
4. Trees are very important data structures in computing.
5. They are suitable for:
 - a. Hierarchical structure representation, e.g.,
 - i. File directory.
 - ii. Organizational structure of an institution.
 - iii. Class inheritance tree.
 - b. Problem representation, e.g.,
 - i. Expression tree.
 - ii. Decision tree.
 - c. Efficient algorithmic solutions, e.g.,
 - i. Search trees.
 - ii. Efficient priority queues via heaps.

AVL TREE

The AVL tree is named after its two inventors, G.M. Adelson-Velsky and E.M. Landis, who published it in their 1962 paper "An algorithm for the organization of information."

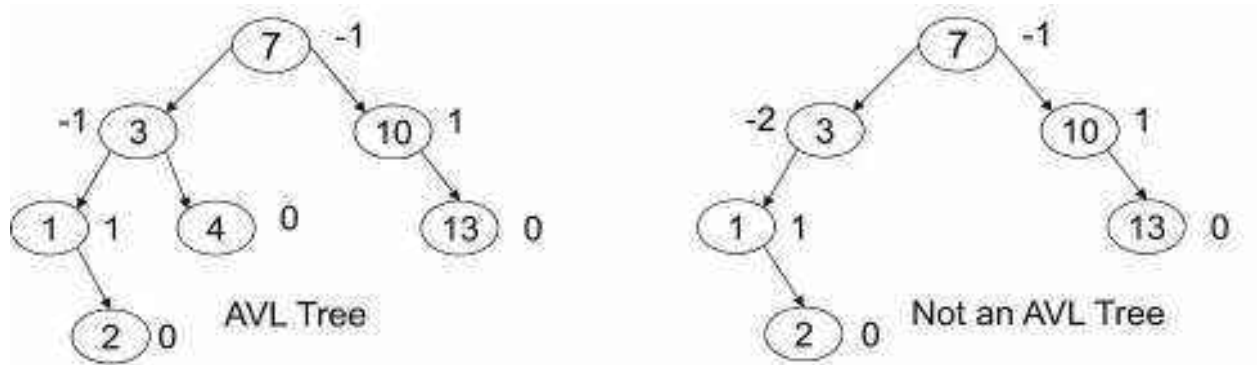
Avl tree is a self-balancing binary search tree. In an AVL tree, the heights of the two child subtrees of any node differ by at most one; therefore, it is also said to be height-balanced.

The balance factor of a node is the height of its right subtree minus the height of its left subtree and a node with balance factor 1, 0, or -1 is considered balanced. A node with any other balance factor is considered unbalanced and requires rebalancing the tree. This can be done by avl tree rotations

Need for AVL tree

- The disadvantage of a binary search tree is that its height can be as large as $N-1$
- This means that the time needed to perform insertion and deletion and many other operations can be $O(N)$ in the worst case
- We want a tree with small height
- A binary tree with N nodes has height at least $\log N$
- Thus, our goal is to keep the height of a binary search tree $O(\log N)$
- Such trees are called balanced binary search trees. Examples are AVL tree, red-black tree.

Thus we go for AVL tree.



HEIGHTS OF AVL TREE

An AVL tree is a special type of binary tree that is always "partially" balanced. The criteria that is used to determine the "level" of "balanced-ness" which is the difference between the heights of subtrees of a root in the tree. The "height" of tree is the "number of levels" in the tree. The height of a tree is defined as follows:

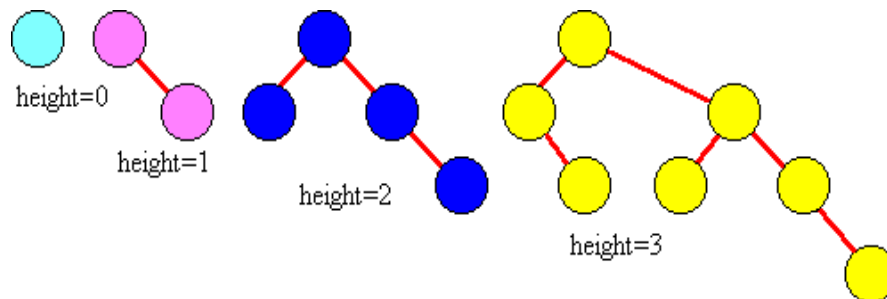
1. The height of a tree with no elements is 0
2. The height of a tree with 1 element is 1
3. The height of a tree with > 1 element is equal to 1 + the height of its tallest subtree.
4. The height of a leaf is 1. The height of a null pointer is zero.

The height of an internal node is the maximum height of its children plus 1.

FINDING THE HEIGHT OF AVL TREE

AVL trees are identical to standard binary search trees except that for every node in an AVL tree, the height of the left and right subtrees can differ by at most 1. AVL trees are HB-k trees (height balanced trees of order k) of order HB-1. The following is the height differential formula:
 $|\text{Height}(Tl) - \text{Height}(Tr)| \leq k$

When storing an AVL tree, a field must be added to each node with one of three values: 1, 0, or -1. A value of 1 in this field means that the left subtree has a height one more than the right subtree. A value of -1 denotes the opposite. A value of 0 indicates that the heights of both subtrees are the same.
EXAMPLE FOR HEIGHT OF AVL TREE



An AVL tree is a binary search tree with a balanced condition.

Balance Factor(BF) = $Hl - Hr$. Hl

=> Height of the left subtree. Hr

=> Height of the right subtree.

If $BF = \{-1, 0, 1\}$ is satisfied, only then the tree is balanced.

AVL tree is a Height Balanced Tree.

If the calculated value of BF goes out of the range, then balancing has to be done.

Rotation :

Modification to the tree. i.e. , If the AVL tree is Imbalanced, proper rotations has to be done.

A rotation is a process of switching children and parents among two or three adjacent nodes to restore balance to a tree.

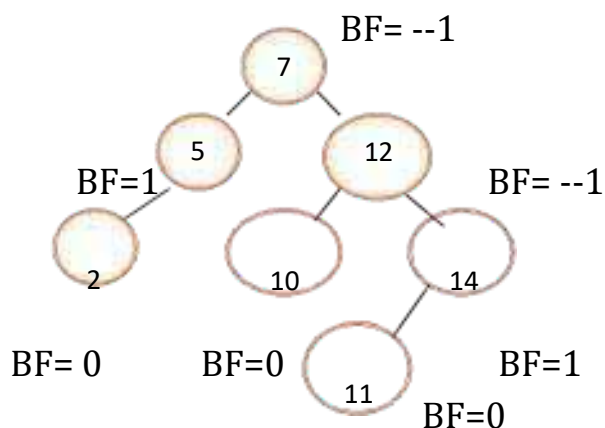
- There are two kinds of single rotation:



An insertion or deletion may cause an imbalance in an AVL tree.

The deepest node, which is an ancestor of a deleted or an inserted node, and whose balance factor has changed to -2 or +2 requires rotation to rebalance the tree.

Balance Factor :



This Tree is an AVL Tree and a height balanced tree.

An AVL tree causes imbalance when any of following condition occurs:

- i. An insertion into Right child's right subtree.
- ii. An insertion into Left child's left subtree.
- iii. An insertion into Right child's left subtree.

iv. An insertion into Left child's right subtree.

These imbalances can be overcome by,

1. **Single Rotation** – (If insertion occurs on the outside,i.e.,LL or RR)

-> LL (Left -- Left rotation) --- Do single Right.

-> RR (Right -- Right rotation) – Do single Left.

2. **Double Rotation** - (If insertion occurs on the inside,i.e.,LR or RL)

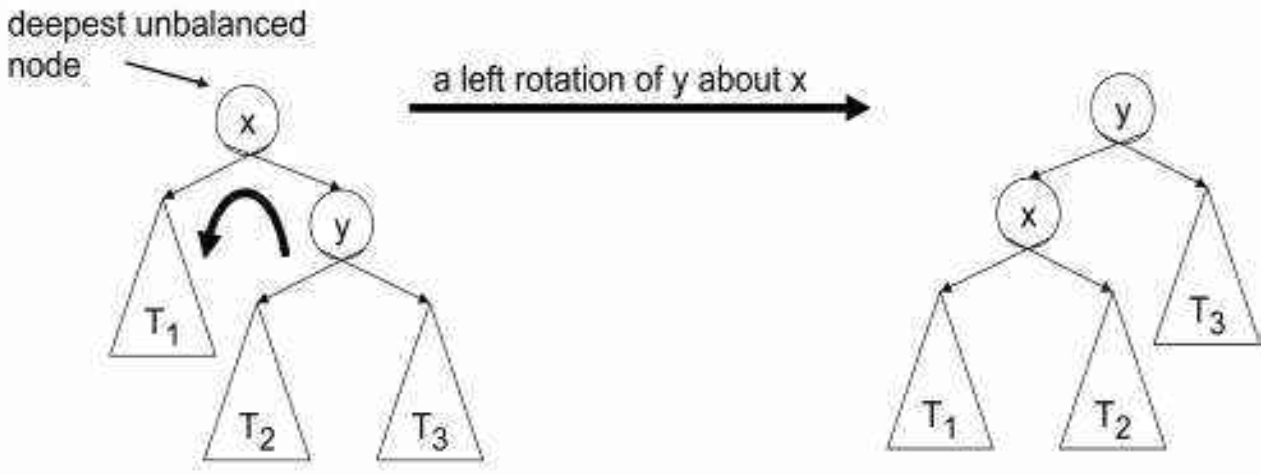
-> RL (Right -- Left rotation) --- Do single Right, then single Left.

-> LR (Left -- Right rotation) --- Do single Left, then single Right.

General Representation of Single Rotation

1. LL Rotation :

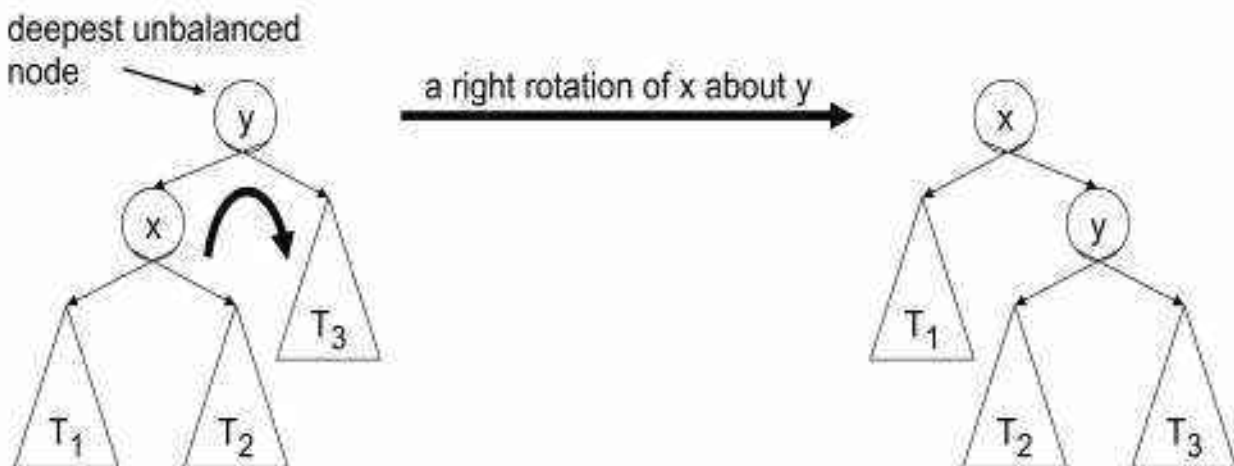
- The right child y of a node x becomes x's parent.
- x becomes the left child of y.
- The left child T_2 of y, if any, becomes the right child of x.



Note: The pivot of the rotation is the deepest unbalanced node

2. RR Rotation :

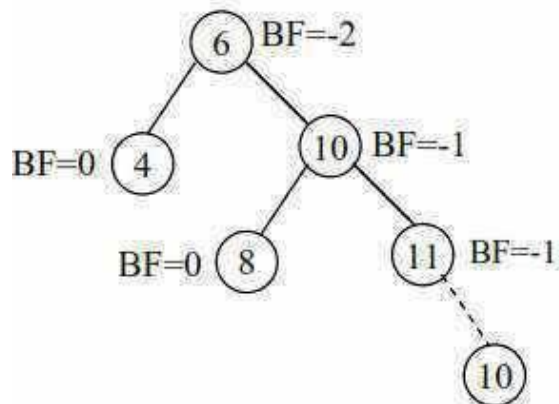
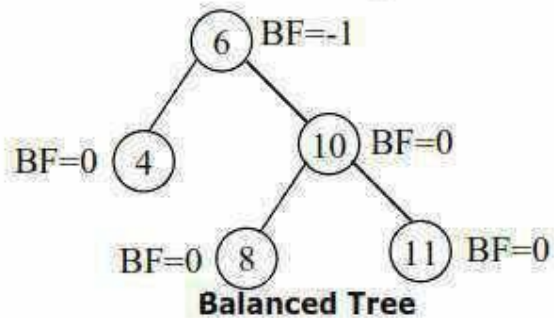
- The left child x of a node y becomes y's parent.
- y becomes the right child of x.
- The right child T_2 of x, if any, becomes the left child of y.



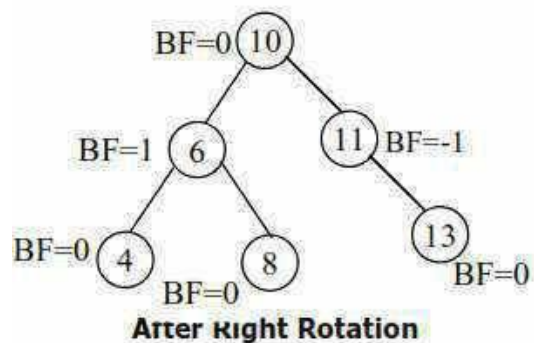
Note: The pivot of the rotation is the deepest unbalanced node

Example:

Consider the following tree which is balanced.

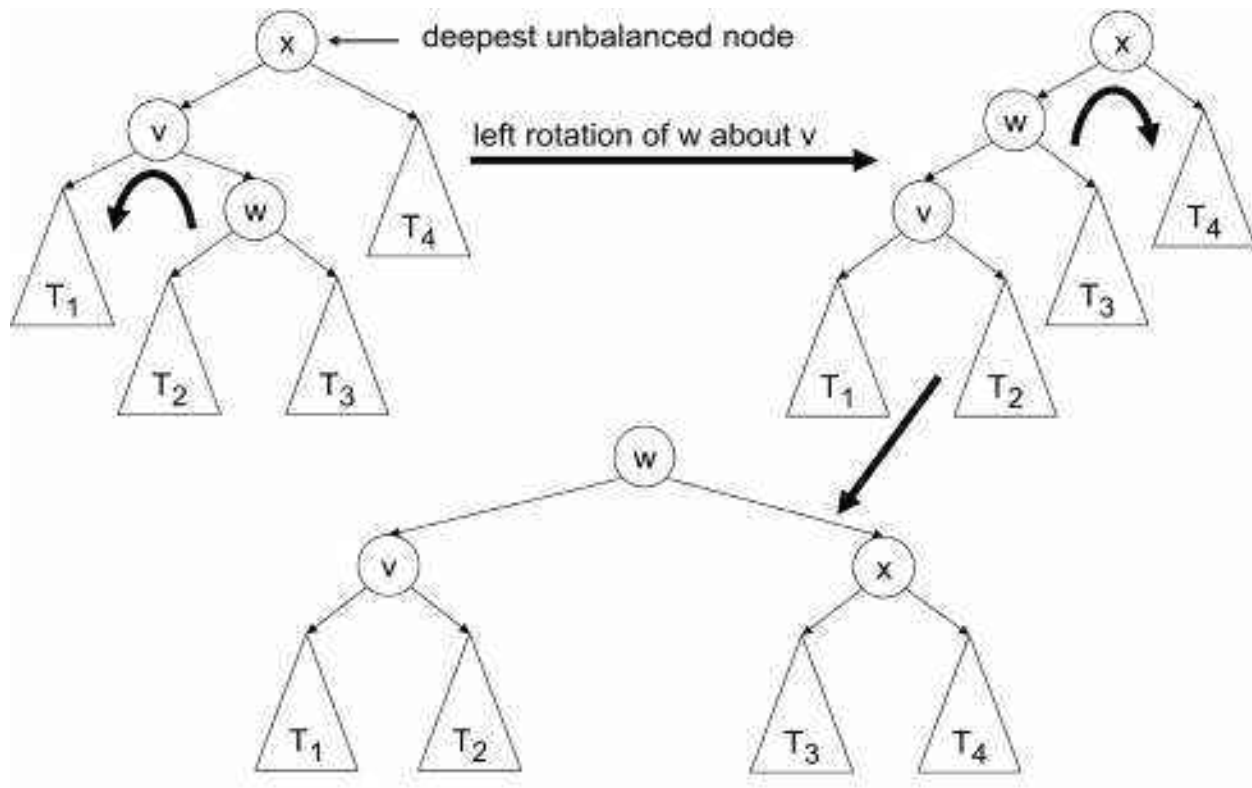


Now insert the value '13' it becomes unbalanced due to the insertion of new node in the Right Subtree of the Right Subtree. So we have to make single Right rotation in the root node.



General Representation of Double Rotation

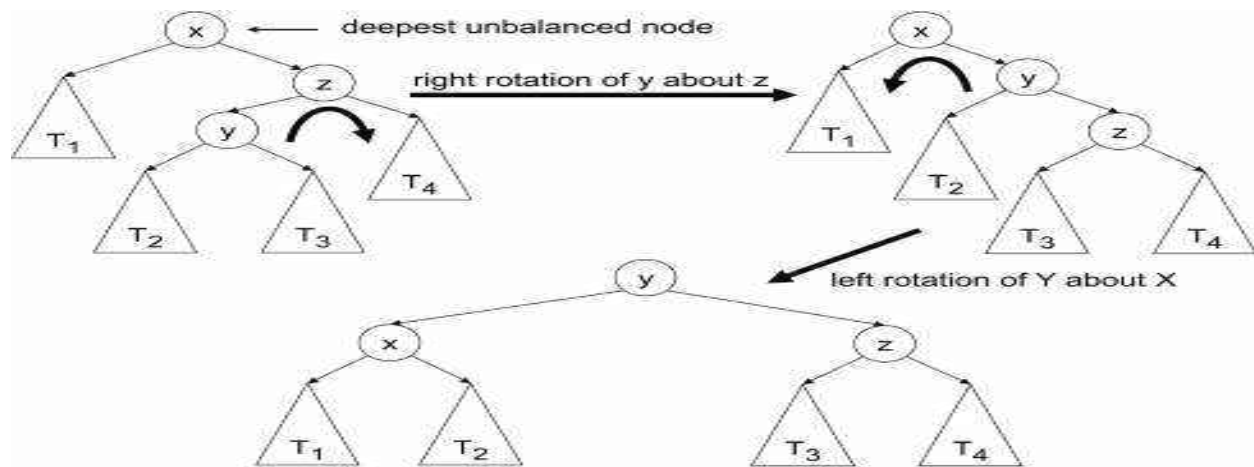
1. LR(Left -- Right rotation):



Note: First pivot is the left child of the deepest unbalanced node; second pivot is the deepest unbalanced node

2. RL(Right -- Left rotation) :

Note: First pivot is the right child of the deepest unbalanced node; second pivot is the deepest unbalanced node



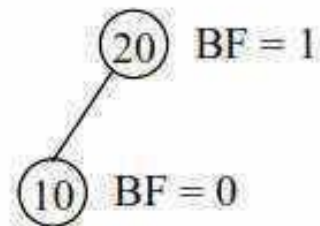
EXAMPLE:

LET US CONSIDER INSERTING OF NODES 20,10,40,50,90,30,60,70 in an AVL TREE

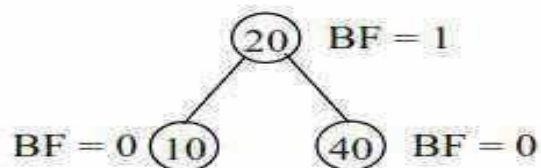
Step 1:(Insert the value 20)



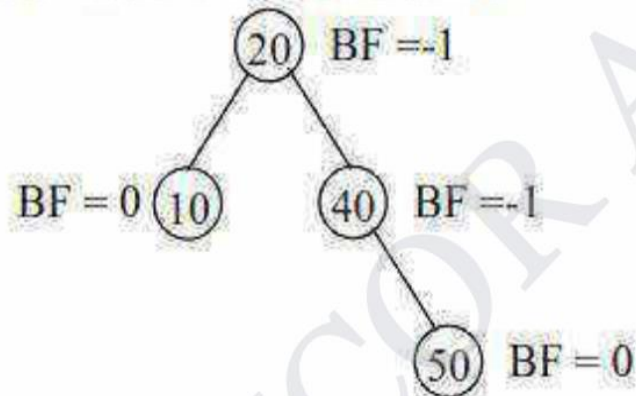
Step 2: (Insert the value 10)



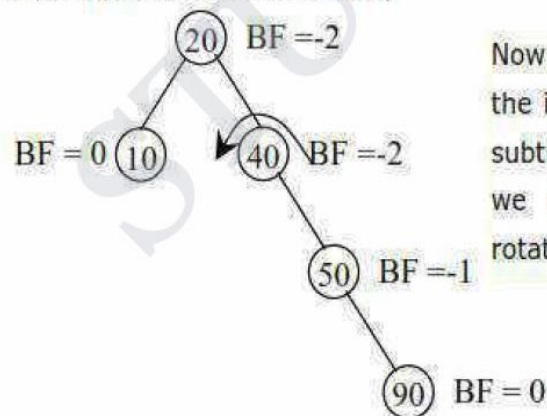
Step 3: (Insert the value 40)



Step 4: (Insert the value 50)

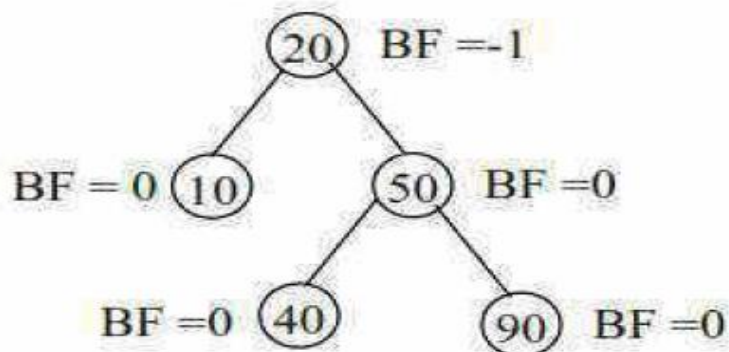


Step 5: (Insert the value 90)



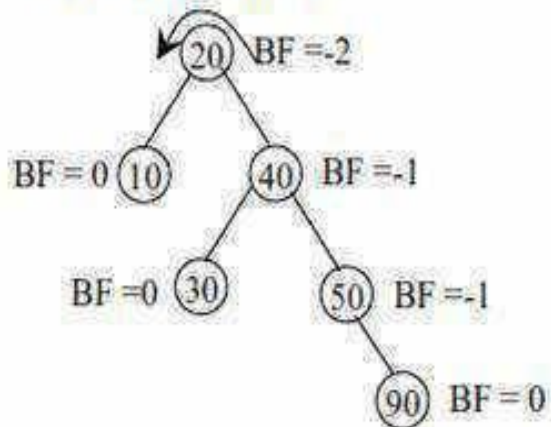
Now the tree is unbalanced due to the insertion of node in the Right subtree of the Right subtree. So we have to make single Right rotation in the node '40'.

After the Right Rotation

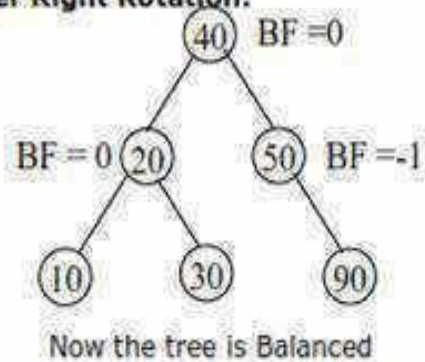


Now the tree is Balanced

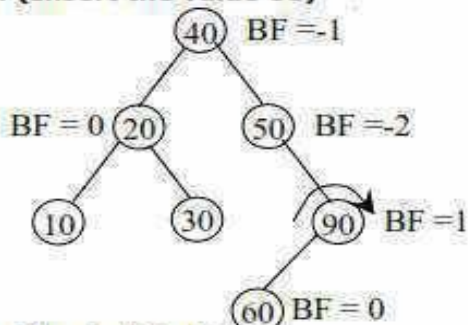
After Left Rotation:



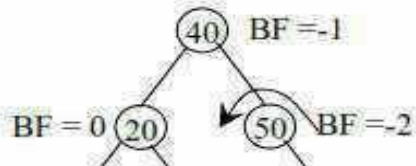
After Right Rotation:



Step 7: (Insert the value 60)

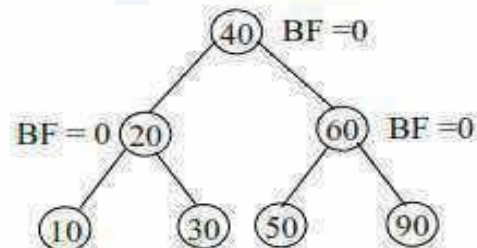


After Left Rotation:



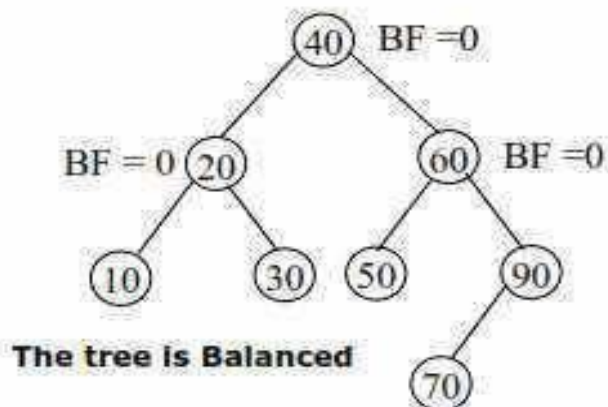
Now the tree at the node '50' is unbalanced due to the insertion of node '60' in the Left subtree of the Right subtree. So we have to make Double rotation first with Left on the node '90' and then with Right on the node '50'.

After Right Rotation:



d

Step 8: (Insert the value 70)



AVL TREE ROUTINES

Creation of AVL Tree and Insertion

Struct avlnode

Typedef struct avlnode *position;

Typedef struct avlnode *avltree;

Typedef int elementtype;

Struct avlnode

{

Elementtype element;

Avltree left;

Avltree right;

Int height;

};

Static int height(position P)

{

If(P==NULL)

return -1;

else

return P-->height;

}

Avltree insert(elementtype X, avltree T)

{

If(T==NULL)

{ /* Create and return a one node tree*/

T= malloc(sizeof(struct avlnode));

If(T==NULL)

Fatalerror("Out of Space");

Else

{

T-->element=X;

```

T-->height=0;
T-->left=T-->right=NULL;
}
}
Else if(X<T-->element)
{
T-->left=Insert(X,T-->left);
If(height(T-->left) - height(T-->right)==2)
If(X<T-->left-->element)
T=singlerotatewithleft(T);
Else
T=doublerotatewithleft(T);
}
Else if(X>T-->element)
{
T-->right=insert(X,T-->right);
If(height(T-->left) - height(T-->right)==2)
If(X>T-->right-->element)
T= singlerotatewithright(T);
Else
T= doublerotatewithright(T);
}
T-->height=max(height(T-->left),height(T-->right)) + 1;
Return T;
}

```

Routine to perform Single Left :

- . This function can be called only if k2 has a left child.
- . Perform a rotate between a node k2 and its left child.
- . Update height, then return the new root.

Static position singlerotatewithleft(position k2)

```

{
Position k1;
k1=k2-->left;
k2-->left=k1-->right;
k1-->right=k2;
k2-->height= max(height(k2-->left),height(k2-->right)) + 1;
k1-->height= max(height(k1-->left),height(k1-->right)) + 1;
return k1; /* New Root */
}

```

Routine to perform Single Right :

```

Static position singlerotationwithright(position k1)
{
position k2;
k2=k1-->right;
k1-->right=k2-->left;
k2-->left=k1;
k2-->height=max(height(k2-->left),height(k2-->right)) + 1;
k1-->height=max(height(k1-->left),height(k1-->right)) + 1;
return k1; /* New Root */
}

```

Double rotation with Left :

```

Static position doublerotationwithleft(position k3)
{
/* Rotate between k1 & k2 */
k3-->left=singlerotatewithright(k3-->left);
/* Rotate between k3 & k2 */
returnsinglerotatewithleft(k3);
}

```

Double rotation with Right :

```
Static position doublerotatewithright(position k1)
{
/* Rotation between k2& k3 */
k1-->right=singlerotatewithleft(k1-->right);
/* Rotation between k1 &k2 */
returnsinglerotatewithright(k1);
}
```

PROBLEMS

APPLICATIONS

AVL trees play an important role in most computer related applications. The need and use of avl trees are increasing day by day. their efficiency and less complexity add value to their reputation. Some of the applications are

- Contour extraction algorithm
- Parallel dictionaries
- Compression of computer files
- Translation from source language to target language
- Spell checker

ADVANTAGES OF AVL TREE

- AVL trees guarantee that the difference in height of any two subtrees rooted at the same node will be at most one. This guarantees an asymptotic running time of $O(\log(n))$ as opposed to $O(n)$ in the case of a standard bst.
- Height of an AVL tree with n nodes is always very close to the theoretical minimum.
- Since the avl tree is height balabced the operation like insertion and deletion have low time complexity.

- Since tree is always height balanced. Recursive implementation is possible.
- The height of left and the right sub-trees should differ by at most 1. Rotations are possible.

DISADVANTAGES OF AVL TREE

- one limitation is that the tree might be spread across memory
- as you need to travel down the tree, you take a performance hit at every level down
- one solution: store more information on the path
- Difficult to program & debug ; more space for balance factor.
- asymptotically faster but rebalancing costs time.
- most larger searches are done in database systems on disk and use other structures

BINARY HEAPS

A **heap** is a specialized complete tree structure that satisfies the *heap property*:

- it is empty *or*
- the key in the root is larger than that in either child and both subtrees have the heap property.
- In general heap is a group of things placed or thrown, one on top of the other.
- In data structures a heap is a binary tree storing keys at its nodes.
- Heaps are based on the concepts of a complete tree

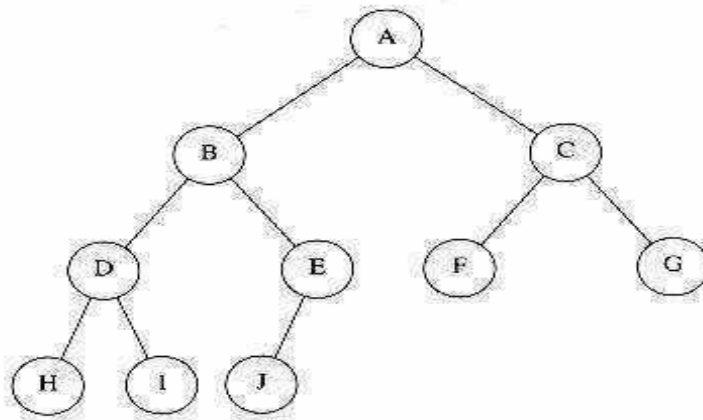
Structure Property :

COMPLETE TREE

A binary tree is completely full if it is of height, h , and has $2^{h+1}-1$ nodes.

- it is empty *or*
- its left subtree is complete of height $h-1$ and its right subtree is completely full of height $h-2$ *or*

- its left subtree is completely full of height $h-1$ and its right subtree is complete of height $h-1$.



A complete binary tree

	A	B	C	D	E	F	G	H	I	J			
0	1	2	3	4	5	6	7	8	9	10	11	12	13

A complete tree is filled from the left:

- all the leaves are on
 - the same level *or*
 - two adjacent ones *and*
- all nodes at the lowest level are as far to the left as possible.

PROCEDURE

INSERTION: Let us consider the element X is to be inserted.

- First the element X is added as the last node.
- It is verified with its parent and adjacent node for its heap property.
- The verification process is carried upwards until the heap property is satisfied.
- If any verification is not satisfied then swapping takes place.
- Then finally we have the heap.

DELETION:

- The deletion takes place by removing the root node.
- The root node is then replaced by the last leaf node in the tree to obtain the complete binary tree.
- It is verified with its children and adjacent node for its heap property.
- The verification process is carried downwards until the heap property is satisfied.
- If any verification is not satisfied then swapping takes place.
- Then finally we have the heap.

1.Heap Order Property :

The property that allows operations to be performed quickly is a heap order property.

Mintree:

Parent should have lesser value than children.

Maxtree:

Parent should have greater value than children.

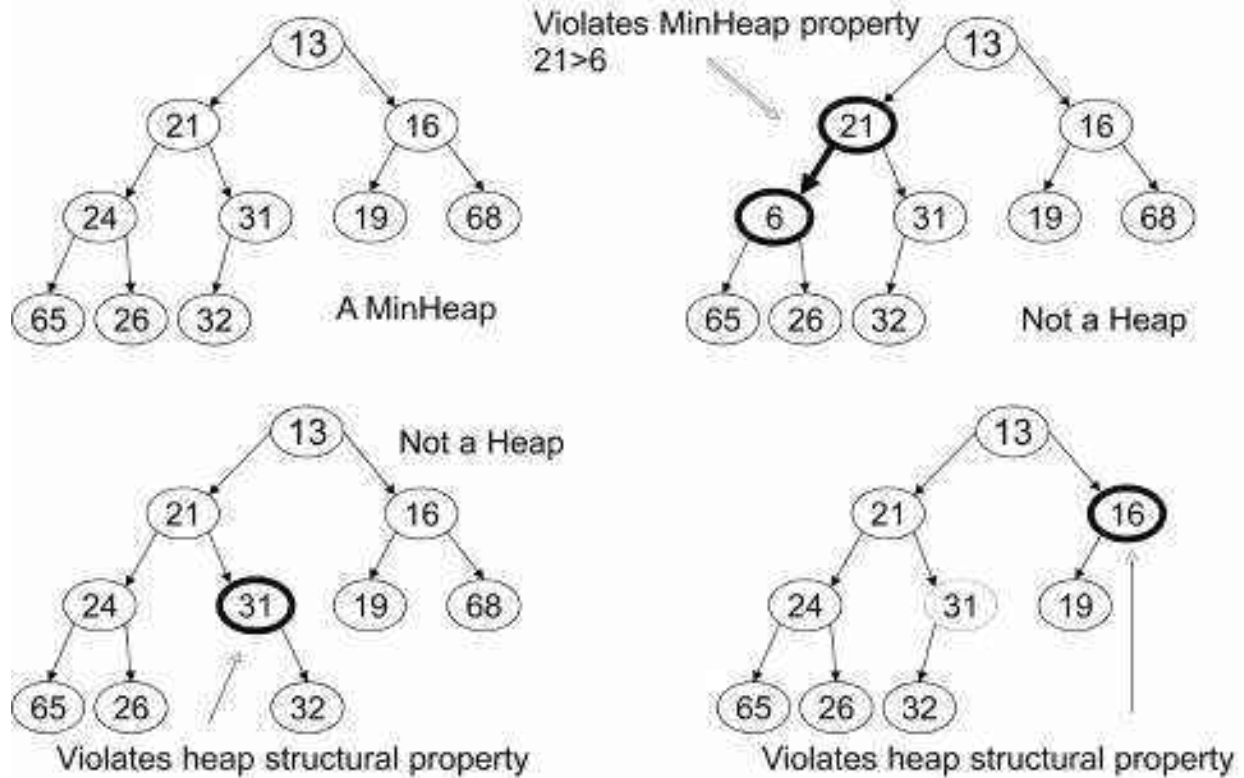
These two properties are known as heap properties

- Max-heap
- Min-heap

Min-heap:

The smallest element is always in the root node. Each node must have a key that is less or equal to the key of each of its children.

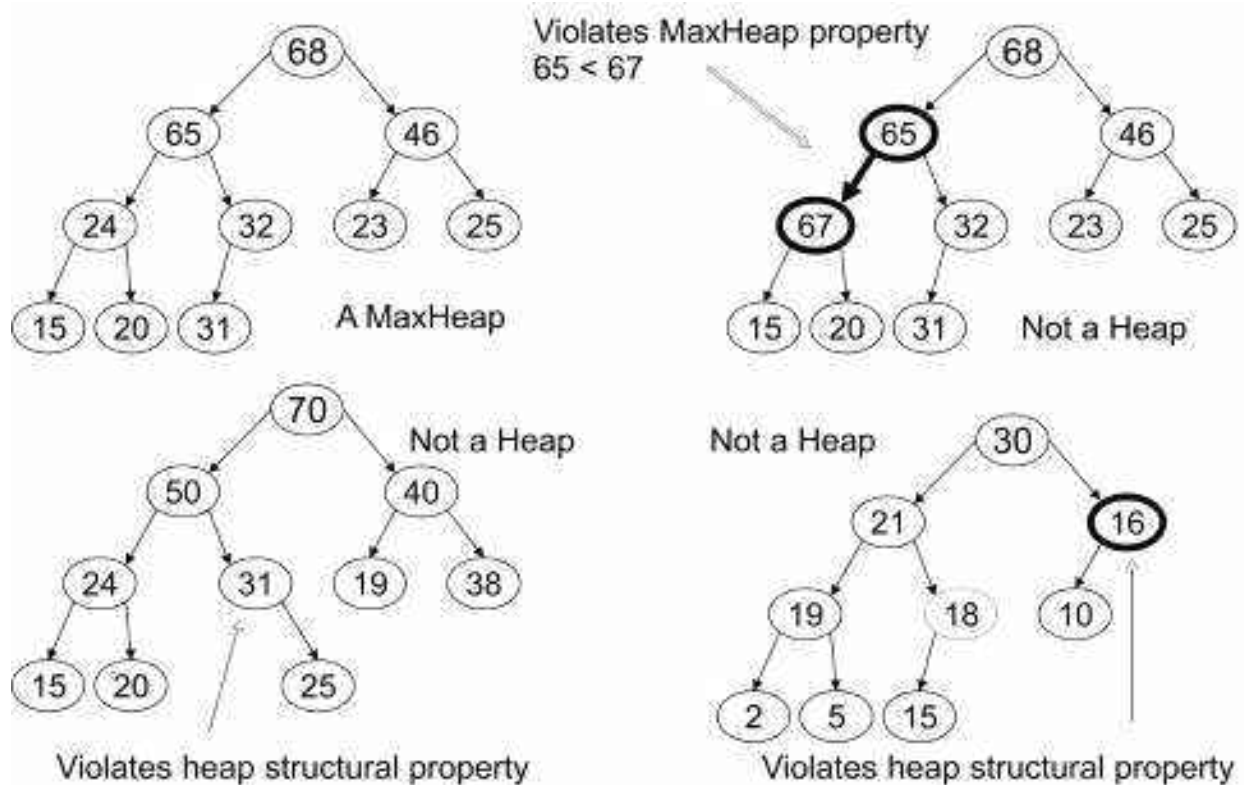
Examples



Max-Heap:

The largest Element is always in the root node. Each node must have a key that is greater or equal to the key of each of its children.

Examples



HEAP OPERATIONS:

There are 2 operations of heap

- Insertion
- Deletion

Insert:

Adding a new key to the heap

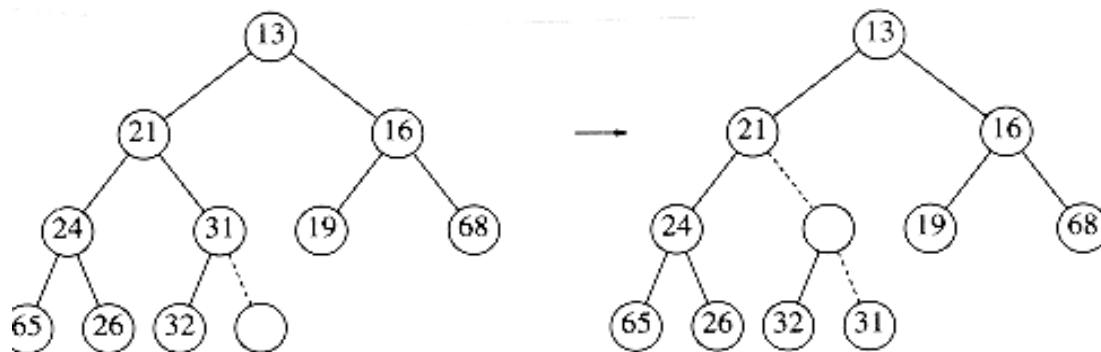
Rules for the insertion:

To insert an element X, into the heap, do the following:

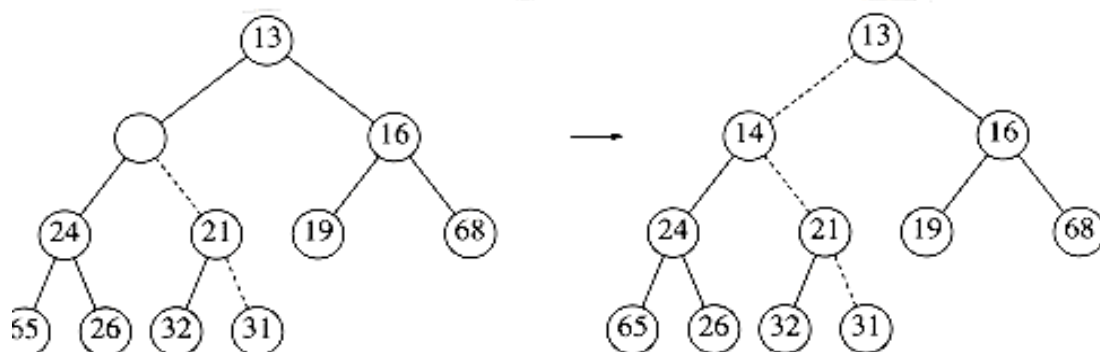
Step1: Create a hole in the next available location , since otherwise the tree will not be complete.

Step2: If X can be placed in the hole, without violating heap order, then do insertion, otherwise slide the element that is in the hole's parent node, into the hole, thus, bubbling the hole up towards the root.

Step3: Continue this process until X can be placed in the hole.



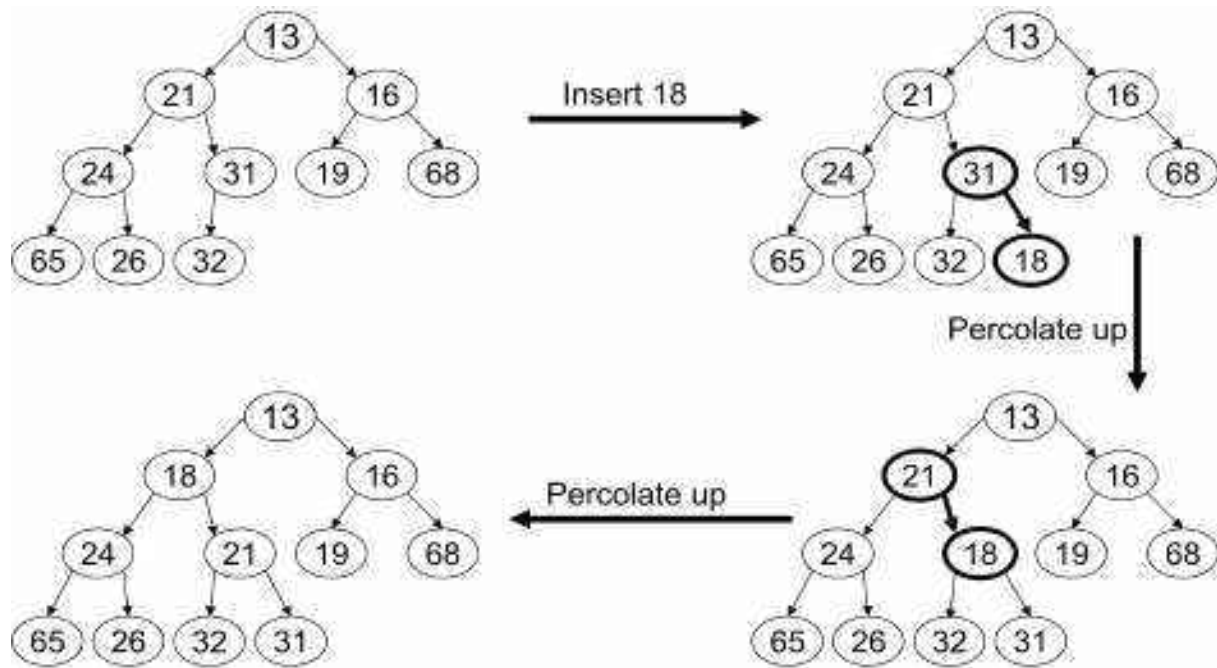
Attempt to insert 14: creating the hole, and bubbling the hole up



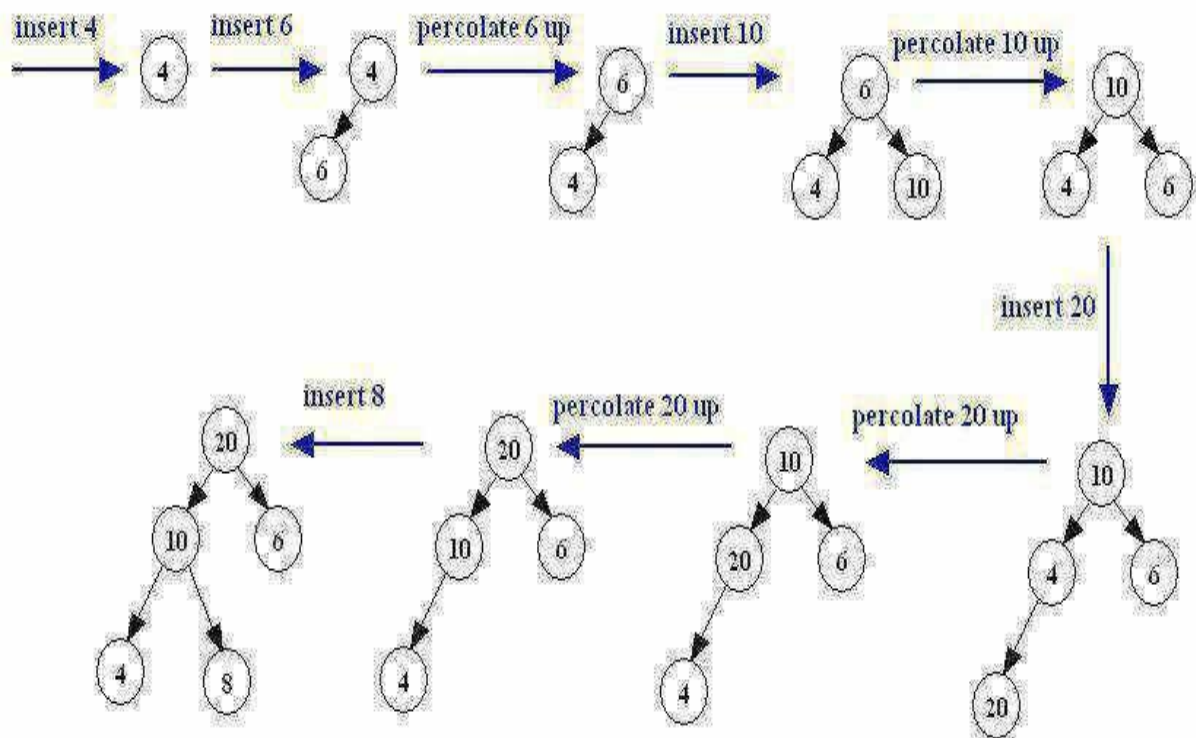
The remaining two steps to insert 14 in previous heap

Example Problem :

1. Insert- 18 in a Min Heap



2. Insert the keys 4, 6, 10, 20, and 8 in this order in an originally empty max-heap



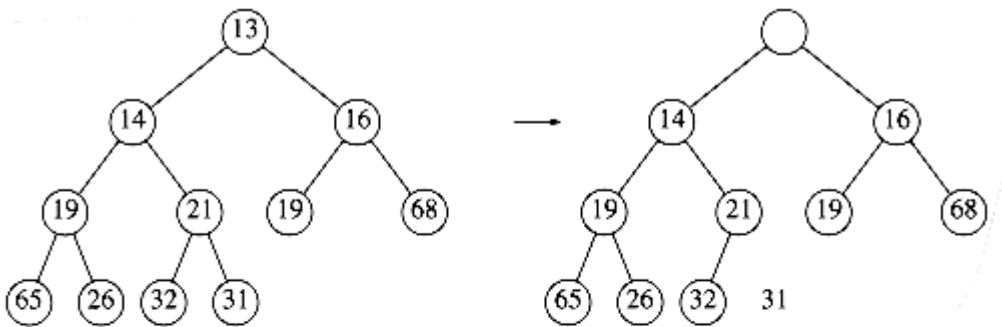
Delete-max or Delete-min:

Removing the root node of a max- or min-heap, respectively

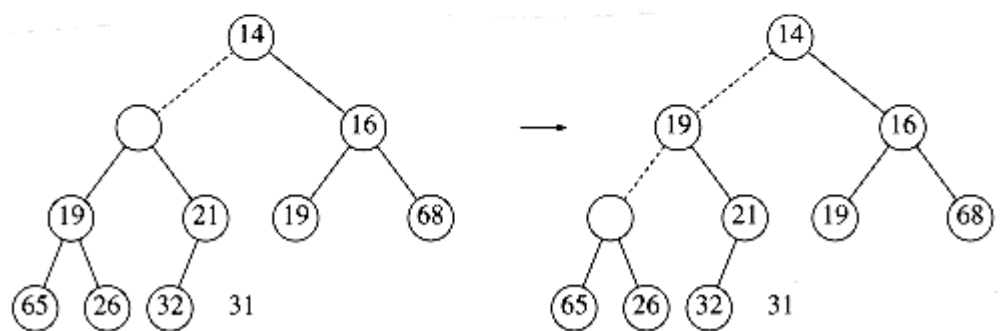
Procedure for Deletemin :

- * Deletemin operation is deleting the minimum element from the heap.
- * In Binary heap | min heap the minimum element is found in the root.
- * When this minimum element is removed, a hole is created at the root.
- * Since the heap becomes one smaller, make the last element X in the heap to move somewhere in the heap.
- * If X can be placed in the hole, without violating heap order property, place it, otherwise slide the smaller of the hole's children into the hole, thus, pushing the hole down one level.
- * Repeat this process until X can be placed in the hole.

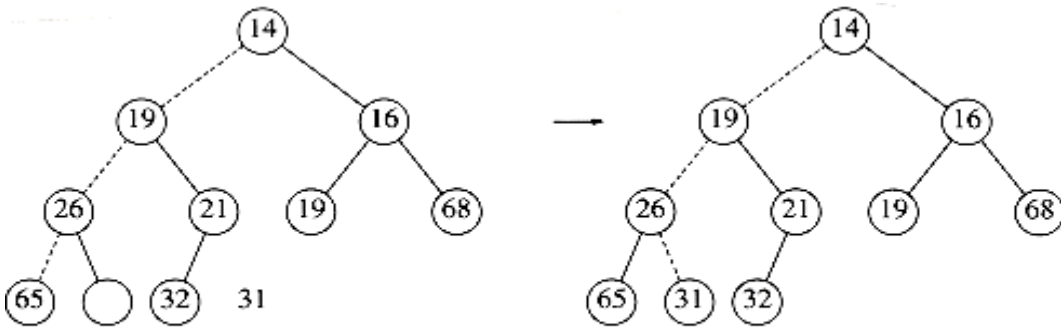
This general strategy is known as Percolate Down.



Creation of the hole at the root



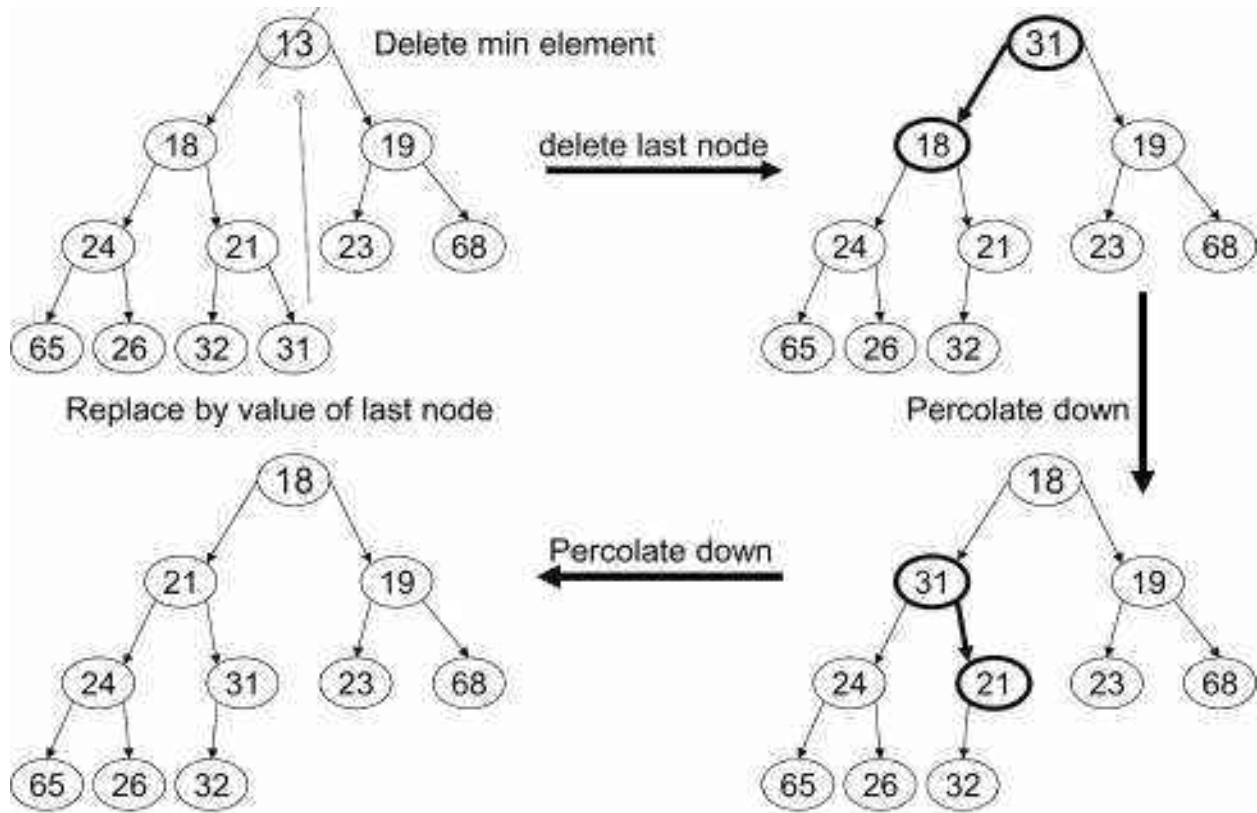
Next two steps in delete_min



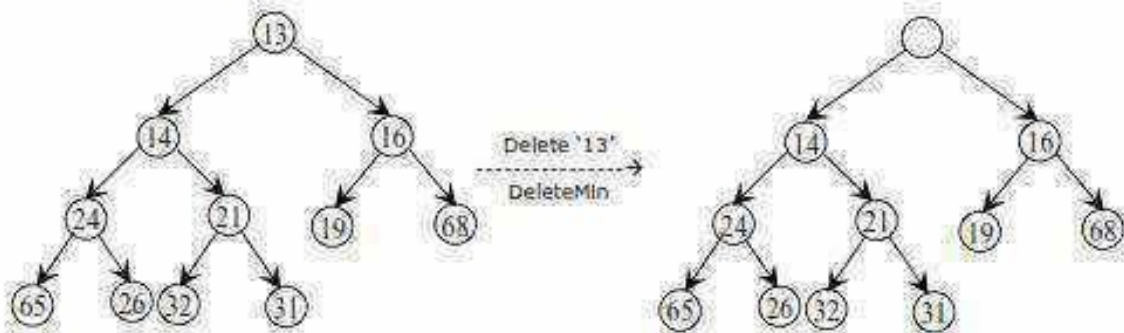
Last two steps in delete_min

EXAMPLE PROBLEMS :

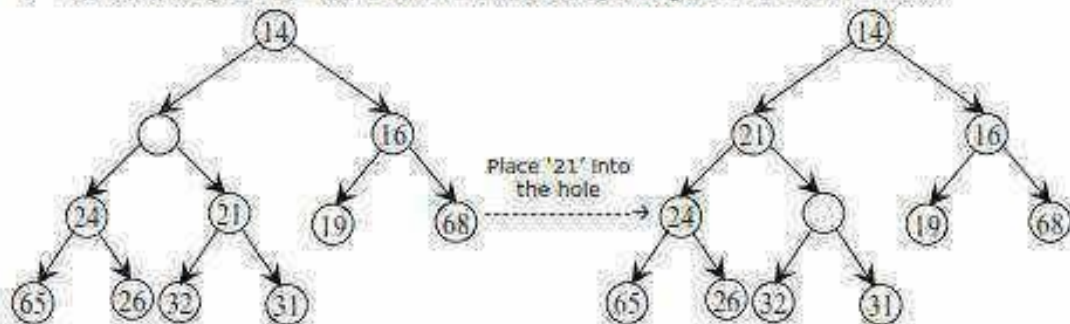
1.DELETE MIN



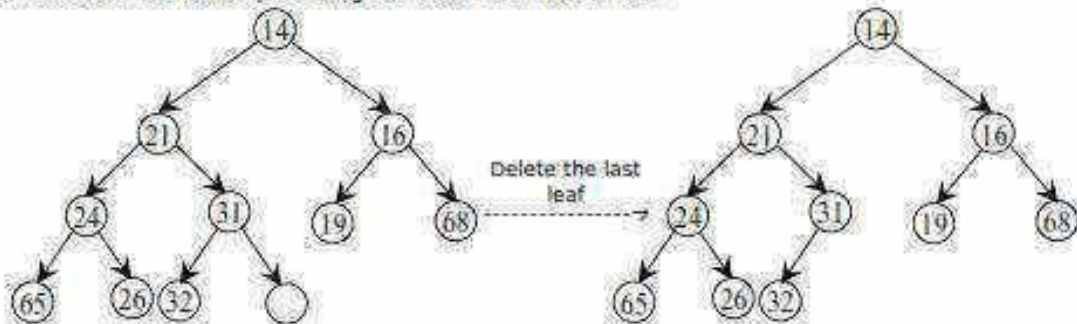
2. Delete Min -- 13



- ❖ The last value cannot be placed in the hole, because this would violate heap order.
- ❖ Therefore, we place the smaller child '14' in the hole, sliding the hole down one level.



Now slide the hole down by inserting the value '31' in to the hole.



BINARY HEAP ROUTINES [Priority Queue]

```
Typedef struct heapstruct *priorityqueue;
```

```
Typedef int elementype;
```

```
Struct heapstruct
```

```
{
```

```
int capacity;
```

```
int size;
```

```
elementtype *element;  
};
```

Declaration of Priority Queue

```
Priorityqueue initialize(int maxelement)  
{  
    Priorityqueue H;  
    If(minpsize<maxelements)  
        Error("Priority queue size is too small");  
    H=malloc(sizeof(struct heapstruct));  
    If(H=NULL)  
        Fatalerror("Out of space");  
    /* Allocate the array plus one extra for sentinel */  
    H-->elements=malloc((maxelements+1)*sizeof(elementtype));  
    If(H-->elements==NULL)  
        Fatalerror("out of space");  
    H-->capacity=maxelements;  
    H-->size=0;  
    H-->elements[0]=mindata;  
    Return H;  
}  
/* H-->elements[0]=sentinelvalue */
```

Insert Routine

```
Void insert(elementtype X, priorityqueue H)  
{  
    int i;  
    if(isfull(H))  
    {  
        Error("Priority queue is full");  
        Return;  
    }  
}
```

```

For(i=++H-->size;H-->elements[i/2]>X;i=i/2)
H-->elements[i]=H-->elements[i/2];
H-->elements[i]=X;
}

```

Delete Routine

```

Elementtype deletemin(priorityqueue H)
{
int i,child;
elementtype minelement,lastelement;
if(isempty(H))
{
Error("Priority queue is empty");
Return H-->element[0];
}
Minelement=H-->element[1];
Lastelement=H-->element[H-->size--];
For(i=1;i*2<=H-->size;i=child)
{
/*Find smaller child */
Child=i*2;
If(child!=H-->size && H-->elements[child++]<H-->elements[child])
{
Child++;
}
/* Percolate one level */
If(lastelement>H-->elements[child])
H-->element[i]=H-->elements[child];
Else
Break;
}
H-->element[i]=lastelement;

```

```
Return minelement;  
}
```

APPLICATIONS

The heap data structure has many applications

- Heap sort
- Selection algorithms
- Graph algorithms

Heap sort :

One of the best sorting methods being in-place and with no quadratic worst-case scenarios.

Selection algorithms:

Finding the min, max, both the min and max, median, or even the k -th largest element can be done in linear time using heaps.

Graph algorithms:

By using heaps as internal traversal data structures, run time will be reduced by an order of polynomial. Examples of such problems are Prim's minimal spanning tree algorithm and Dijkstra's shortest path problem.

ADVANTAGE

The biggest advantage of heaps over trees in some applications is that construction of heaps can be done in linear time.

- It is used in
 - Heap sort
 - Selection algorithms
 - Graph algorithms
 -

DISADVANTAGE

Heap is expensive in terms of

- safety
- maintenance
- performance

Performance :

Allocating heap memory usually involves a long negotiation with the OS.

Maintenance:

Dynamic allocation may fail; extra code to handle such exception is required.

Safety :

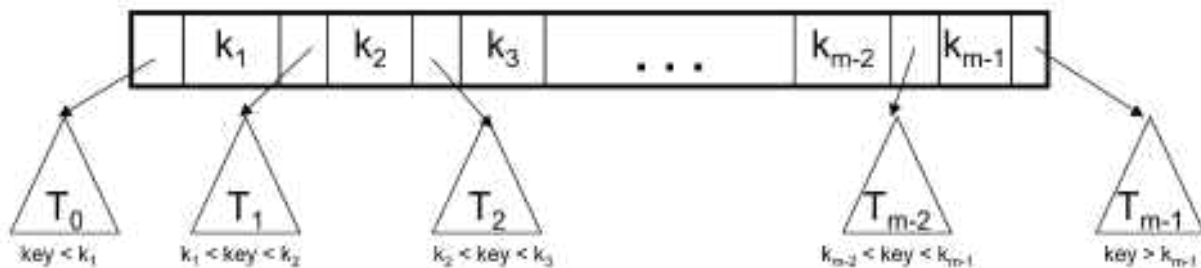
Object may be deleted more than once or not deleted at all .

B-TREES

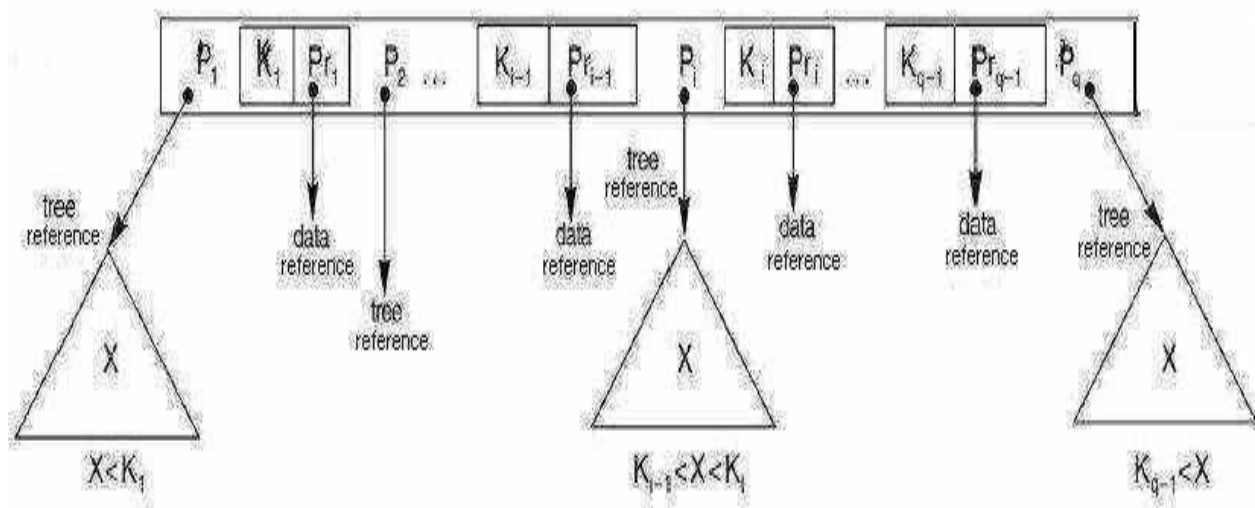
Multi-way Tree

A multi-way (or m-way) search tree of order m is a tree in which

- Each node has at-most m subtrees, where the subtrees may be empty.
- Each node consists of at least 1 and at most m-1 distinct keys
- The keys in each node are sorted.



- The keys and subtrees of a non-leaf node are ordered as:
 - $T_0, k_1, T_1, k_2, T_2, \dots, k_{m-1}, T_{m-1}$ such that:
- All keys in subtree T_0 are less than k_1 .
- All keys in subtree $T_i, 1 \leq i \leq m - 2$, are greater than k_i but less than k_{i+1} .
- All keys in subtree T_{m-1} are greater than k_{m-1}

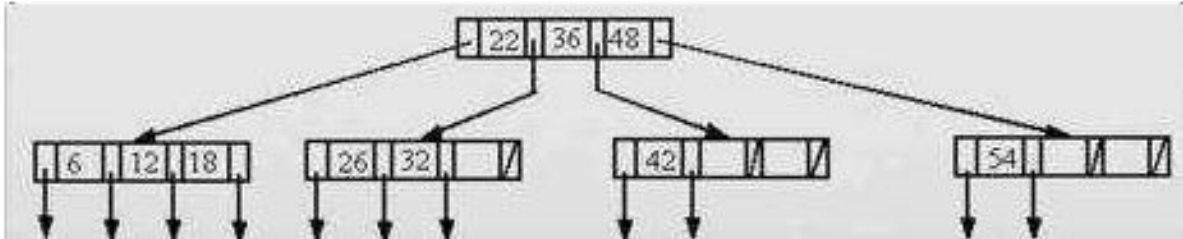


A **B-tree** of order m (or branching factor m), where $m > 2$, is either an empty tree or a multiway search tree with the following properties:

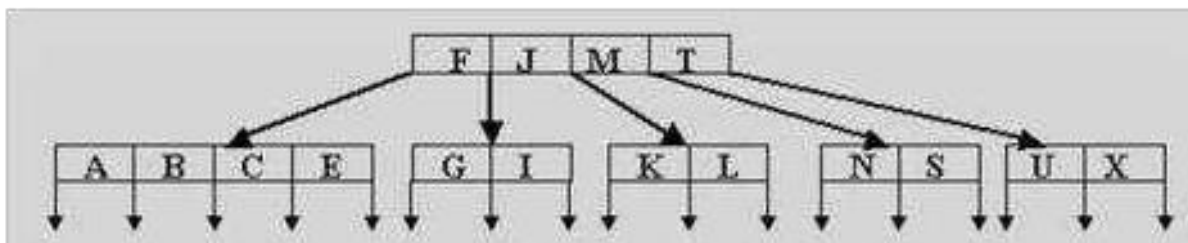
- The root is either a leaf or it has at least two non-empty subtrees and at most m non-empty subtrees.
- Each non-leaf node, other than the root, has at least $\lceil m/2 \rceil$ non-empty subtrees and at most m non-empty subtrees. (Note: $\lceil x \rceil$ is the lowest integer $> x$).
- The number of keys in each non-leaf node is one less than the number of non-empty subtrees for that node.
- All leaf nodes are at the same level; that is the tree is perfectly balanced.

B-Tree Examples

Example: A B-tree of order 4



Example: A B-tree of order 5



Insertion in B-Trees

OVERFLOW CONDITION:

A root-node or a non-root node of a B-tree of order m overflows if, after a key insertion, it contains m keys.

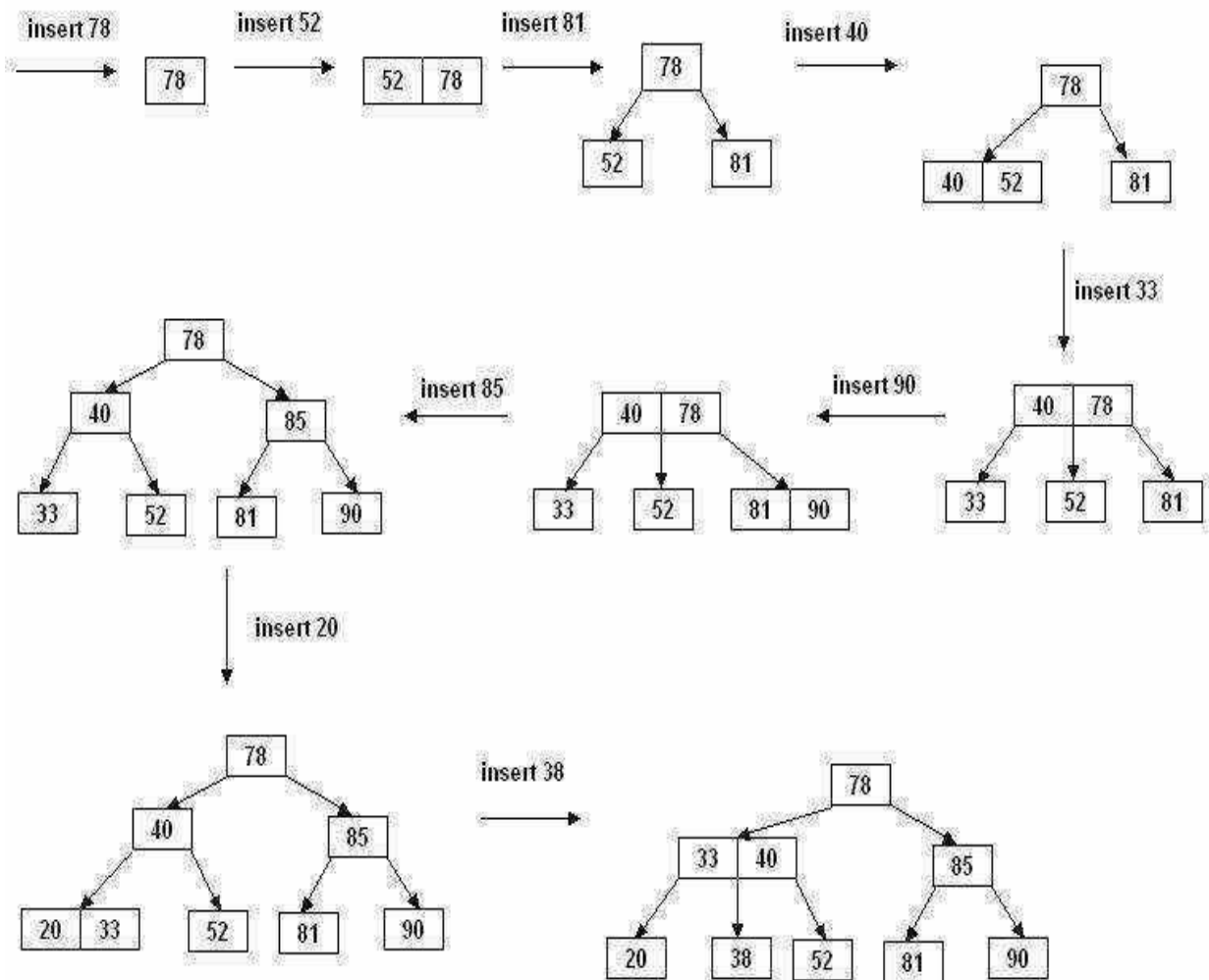
Insertion algorithm:

If a node overflows, split it into two, propagate the "middle" key to the parent of the node. If the parent overflows the process propagates upward. If the node has no parent, create a new root node.

- Note: Insertion of a key always starts at a leaf node.

Insertion in a B-tree of odd order

Example: Insert the keys 78, 52, 81, 40, 33, 90, 85, 20, and 38 in this order in an initially empty B-tree of order 3

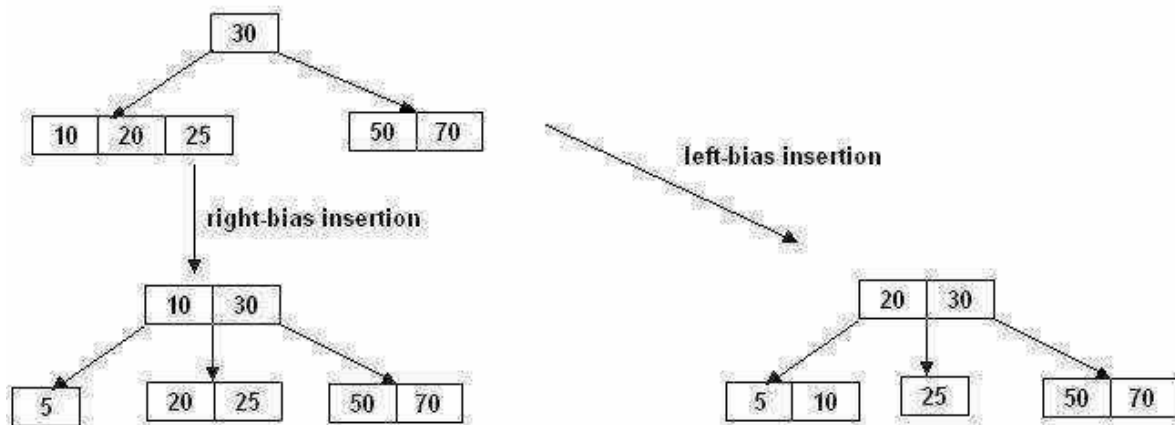


Insertion in a B-tree of even order

At each node the insertion can be done in two different ways:

- **right-bias:** The node is split such that its right subtree has more keys than the left subtree.
- **left-bias:** The node is split such that its left subtree has more keys than the right subtree.

Example: Insert the key 5 in the following B-tree of order 4:



ADVANTAGES:

- B-trees are suitable for representing huge tables residing in secondary memory because:
 1. With a large branching factor m , the height of a B-tree is low resulting in fewer disk accesses.
 2. The branching factor can be chosen such that a node corresponds to a block of secondary memory.
 3. The most common data structure used for database indices is the B-tree. An **index** is any data structure that takes as input a property (e.g. a value for a specific field), called the search key, and **quickly** finds all records with that property.

Note: As m increases the amount of computation at each node increases; however this cost is negligible compared to hard-drive accesses.